Automated Content Recommendation Feedback Dashboard

A DevOps project delivered around one working web application, with the whole build-test-package-deploy-provision chain automated.

Explanation of the work done, and a step-by-step laboratory demonstration guide

Item	Detail
Project	CRFD - Automated Content Recommendation Feedback Dashboard
Version	1.0.0 (release baseline tagged v1.0.0; MVP baseline tagged v0.1.0-mvp)
Type	DevOps / Agile group project - 15-week MVP plus delivery toolchain
Repository	content-recommendation-feedback-dashboard (Git, 22 branches, 19 reviewed pull requests, 2 annotated tags)
Application	Java 21, Spring Boot 3.3.5, Maven wrapper 3.9.16, H2 (file) with a MySQL profile, embedded Tomcat fat JAR + a WAR profile
Toolchain	Git/GitHub workflow, Jenkins (config as code), Selenium 4, Docker, Ansible, Nginx
Verification	32 unit + API tests, 5 Selenium journeys, 11/11 deployment smoke checks, idempotent provisioning (changed=0), rollback proven
This document	What was asked, what we built, how we planned and ran it, task by task, then exactly what to show in the lab

How to use this document: Section 1 answers "what was asked", Section 2 "what we built", Section 3 "how we planned it", Section 4 the week-wise task list, Section 5 the task-by-task record of work done and the evidence for each, and Section 6 is the demonstration run sheet to read from during the lab. Sections 7 to 11 are the evidence index, troubleshooting, limitations and viva answers.

Contents

Section	What it is for
1. What was asked	The brief, requirement by requirement, and where each requirement is satisfied
2. What we built	The product, the stack and the architecture, in one place
3. How we planned and ran it	Agile plan, Definition of Ready/Done, sprints, velocity, DevOps lifecycle
4. Week-wise task plan	Weeks 1-15 with the stories delivered in each
5. Task-by-task record	Each piece of work: what was asked, what was done, evidence
6. What to show in the lab	The step-by-step demonstration run sheet, with expected output
7. Evidence index	Every file in docs/evidence and what it proves
8. Troubleshooting	The failures that actually happened, and their fixes
9. Limitations and future work	Stated honestly, first rather than last
10. Viva - questions and answers	Short defensible answers to the questions this project invites
11. Appendix	Ports, URLs, configuration, command cheat sheet, key revisions

1. What was asked

1.1 The brief

The brief asked for a 15-week MVP web application plus the complete DevOps toolchain around it. Restated requirement by requirement, without softening any of it:

1. A working MVP web application covering: data/event entry, a searchable dashboard, summary indicators, status drill-down, and an alert/exception view - with the scope frozen.
2. A frozen MVP scope expressed as an Agile backlog, sprint plan, Definition of Done, and a DevOps lifecycle diagram.
3. A chosen technology stack, build tool (Maven, Gradle or Ant), database, and a Tomcat/Nginx deployment target, with a working local setup.
4. Git and GitHub: README, .gitignore, folder structure, issue templates, branch naming rules, meaningful commits, feature branches, pull requests with review, a merge conflict created and resolved, tagging and a release baseline.
5. Jenkins installed and configured, pointed at the repository, building on a commit (polling/webhook), with an archived build artefact.
6. A Jenkinsfile with checkout / build / package / deploy stages and at least one parameterised environment setting.
7. Selenium: 3-5 critical user journeys, run locally through Maven, screenshots on failure, integrated into Jenkins, a failed test stopping the deployment, and a deliberately introduced defect fixed and re-run.
8. Docker: a Dockerfile, build/tag/run, port mapping, logs, and the stop/restart/remove lifecycle; Jenkins building a versioned image, publishing it to Docker Hub or a local registry, and deploying a fresh container.
9. Provisioning with Puppet or Ansible: a clean node brought to a working state, idempotency evidence, a health check, and rollback/recovery.
10. Finally: an end-to-end workflow, a complete report, a troubleshooting guide, limitations, future enhancements, and a presentation/viva.

1.2 Requirement to evidence: where each one is satisfied

What the brief asks for	Where it is delivered	Evidence to show
Working MVP web application	src/main/java (domain, service, web), src/main/resources/static (dashboard)	The running dashboard; the 32 automated tests; Swagger UI
Frozen scope, backlog, sprint plan, DoD, DevOps lifecycle diagram	docs/02-agile-plan.md; docs/01-problem-definition-and-scope.md	Section 4 of this document; the lifecycle diagram in docs/02
Stack, build tool, database, Tomcat/Nginx target, working local setup	docs/03-srs-and-architecture.md; README.md; pom.xml; Dockerfile; docker-compose.yml	Section 2.3; `mvnw clean package`; the Nginx-fronted Compose stack
Git/GitHub: README, .gitignore, structure, templates, branching, commits, PRs, conflict, tags	README.md, BRANCHING.md, CONTRIBUTING.md, .github/, docs/06, docs/07, docs/pull-requests/README.md	`git log --graph`; the merge commit with two parents; the two annotated tags; the conflict evidence file

Jenkins installed and configured, GitHub repo, build on commit, archived artefact	infra/jenkins/ (Compose + JCasC + plugins.txt + seed job), docs/08	Jenkins on http://localhost:8085; the job created by the seed job; the archived-artefact table
Jenkinsfile: checkout/build/package/deploy, parameterised environment	Jenkinsfile (10 stages), docs/09, docs/13	The Jenkinsfile in the repository; the ENVIRONMENT and CRFD_* parameters
Selenium 3-5 journeys, via Maven, screenshots on failure, in Jenkins, blocks deploy, defect fixed and re-run	src/test/java/.../selenium/, docs/10, docs/11	5/5 journeys green; the failure screenshot triple; the regression tests failing before the fix
Docker: Dockerfile, build/tag/run, port mapping, logs, stop/restart/remove	Dockerfile, scripts/docker-lifecycle.*, docs/12	scripts/docker-lifecycle.ps1 transcript; docker-inspect.txt
Jenkins builds a versioned image, publishes it, deploys a fresh container	Jenkinsfile stages 4, 7, 8; infra/jenkins/; docs/13	The three tags per build; BUILD_REVISION baked into the image; the deploy stage
Ansible: clean node, idempotency, health check, rollback/recovery	infra/ansible/, scripts/ansible-demo.*, docs/14	ansible-summary.txt: changed=0 on the repeat run, and the rollback line
End-to-end workflow, report, troubleshooting, limitations, future work, viva	docs/15, docs/16, docs/17, docs/18, this document	The walkthrough in docs/15; the limitations list in docs/16

1.3 Two clarifications we made and recorded

The original brief lists "MVP Completion and Git Collaboration" and "Jenkins-Docker Continuous Deployment" as two deliverables each. We merged each duplicate into a single deliverable (docs/07 and docs/13) rather than delivering the same thing twice, and recorded the decision so the mapping stays explicit.

Explicit feedback was added as a sixth capability (CRFD-106). The brief lists five capabilities; rating capture is arguably implied by the phrase "feedback dashboard", but a frozen scope that grows quietly is not frozen. It was added with a change request, given its own story, tests and journey coverage, and written down.

1.4 What we deliberately did NOT build

Naming the exclusions is part of defining the scope. Each was considered and deferred, with the reason recorded in docs/01:

Excluded	Why
Authentication and authorisation	The MVP runs on a trusted internal network; the capacity was needed by the six capabilities. The alert "owner" is free text.
A real recommendation model server	The thing under test is the feedback pipeline, not the model. Serving latency is simulated so the latency thresholds are genuinely exercised.
Multi-tenancy	No requirement exists yet; it would change every table

	and every query.
Streaming ingestion (Kafka and similar)	An HTTP ingest endpoint proves the same pipeline semantics with far less operational surface, and keeps the MVP runnable from one JAR.
A full-text search engine	The MVP search is a substring match over indexed columns - correct and adequate at this volume.
Mobile client	The dashboard is a desktop tool; a responsive layout is sufficient.
Alert delivery by email/Slack/PagerDuty	Alerts are surfaced in the dashboard and over the API. The delivery channel is an integration, not a capability.

2. What we built

2.1 The product in one paragraph

An operator opens the dashboard and immediately sees whether the recommendation pipeline is healthy: how many events arrived, how many were processed, the p95 serving latency, how many exceptions are open, and a deep-health status. They can search events by free text and facets, export the current filter to CSV, open any event to see its full status-transition timeline and the exceptions it raised, manually move an event's status, submit a new feedback event or rating, and work an exception queue by acknowledging and resolving alerts. Six capabilities, frozen at the start of the project, and nothing else.

2.2 The six frozen capabilities

ID	Capability	What it does	Acceptance, in one line
CRFD-101	Data / event entry	Publish recommendation feedback events; bulk ingest; classification	POST returns 201 + Location; a duplicate eventId returns 409 AND raises an alert; a payload missing itemId is QUARANTINED, not rejected
CRFD-102	Searchable dashboard	Free-text search, facets, paging, sorting, CSV export	Filter by status/source/surface/type/item/user; CSV export capped at 5000 rows
CRFD-103	Summary indicators	Totals, rates, p95 latency, time series, breakdowns	Aggregate queries, not in-memory loops; hourly/daily throughput
CRFD-104	Status drill-down	Event detail, immutable audit timeline, manual transition, reprocess	Every transition appends an audit entry ordered by (createdAt, id)
CRFD-105	Alert / exception view	Automatic raising from threshold breaches; queue with acknowledge/resolve	A repeat breach of the same code on the same event does not open a second alert
CRFD-106	Explicit feedback (added by change request)	Rating 1-5 with a reason; listing by item	A low rating raises a RATING_DROP alert; feedback is not idempotent - the newest rating wins

2.3 The stack, and why each choice was made

Decision	Chosen	Rejected alternatives	Why
Language and framework	Java 21 + Spring Boot 3.3.5	Node/Express, Python/FastAPI	Spring Data JPA gives aggregate projections and specification-based search; Java 21 records remove DTO boilerplate; existing skill set
Build tool	Maven with the wrapper	Gradle, Ant	Profiles express "JAR or

	committed (3.9.16)		WAR or MySQL" declaratively, which is exactly the shape of the requirement; the wrapper pins one version for laptop, CI and Docker
Database	H2 in file mode by default, MySQL 8 as a profile	MySQL everywhere, H2 in memory	A reviewer can run the product with zero installation, and file mode survives a restart so drill-down still has data
Deployment artefact	Fat JAR with embedded Tomcat, plus a war profile	WAR only, external container always	One artefact is the cheapest thing to build, ship and health-check; the WAR profile proves the app is not coupled to its embedded container
Front end	Vanilla ES2020 + CSS served from static/ (no bundler)	React + Vite	A bundler adds a Node toolchain, a second build step and a hash manifest to the deploy; the artefact stays one JAR
Data model	One wide event table	Normalised payload table, event sourcing	Writes are the hot path and the dashboard is one indexed search over the same rows
Health model	Readiness (/health) and the business SLO (/health/slo) as separate endpoints	One endpoint that is 503 whenever anything is degraded	Conflating them means a business-metric dip takes a healthy node out of rotation - which is exactly what happened twice before it was fixed
Jenkins	Configuration as code (Compose + JCasC + plugins.txt + seed job)	Clicking through the Jenkins UI	A hand-configured controller cannot be reviewed, reproduced or rebuilt

2.4 Architecture

One artefact, three strict layers, and one database. Controllers never touch repositories, and services never return entities to the web layer, so the seams for a future split are already in place.

```
  browser  ->  Nginx (:80 or :8085-side proxy)  ->  Spring Boot application  ->  H2 file /
MySQL

  web layer        EventController  MetricsController  AlertController
                   FeedbackController  HealthController  SimulationController
                   GlobalExceptionHandler  ->  one uniform ApiError contract
  service layer    EventService            ingest / search / export / detail / transition
                   EventProcessingService  the pipeline: score, transition, audit
```

```
                  PipelineScheduler       dispatches processing in its own transaction
                  AlertService            raise / suppress / acknowledge / resolve
                  MetricsService          summary indicators, time series
                  FeedbackService         explicit feedback
                  HealthService           readiness + SLO evaluation
persistence       RecommendationEventRepository  AlertRepository
                  AuditEntryRepository           FeedbackRepository
database          H2 file mode (default)   |   MySQL 8 (mysql profile)
```

Three deployment shapes, the same code

Shape	Artefact	Container	Proxy	Command
A. Embedded Tomcat (default)	executable fat JAR	embedded	none	java -jar app.jar
B. Docker + Nginx (what the pipeline deploys)	image built from the same JAR	Docker, non-root	Nginx container	docker compose up -d --build
C. External Tomcat (the brief's target)	WAR	Tomcat 10.1	optional	mvnw -P war clean package then drop crfd.war into webapps/

Nginx is in front in shape B for three reasons: it terminates the client connection so the application container can be replaced without the entry point changing, it exposes /healthz for deployment tooling, and it applies rate limiting to the ingest path - concerns that do not belong inside the application.

2.5 Data model

Table	Holds	Why it exists
recommendation_event	The event itself: business key, user, item, source, surface, type, score, simulated latency, status, correlation id	The one wide table the dashboard searches, summarises and drills into
event_audit	One row per status transition, with who/why/when	Makes the timeline immutable: the drill-down shows the history, not the current state
alert	Exception rows: code, severity, status, event reference, owner, timestamps	The queue an SRE works; suppression is per (code, eventRef) while OPEN
recommendation_feedback	Explicit ratings 1-5 with a reason and the submitter	Quality that is stated by the user rather than inferred from behaviour

2.6 The API surface

Area	Endpoints
Events (CRFD-101/102/104)	POST /api/v1/events, POST /api/v1/events/bulk, GET /api/v1/events, GET /api/v1/events/export, GET

	/api/v1/events/{eventId}, PATCH /api/v1/events/{eventId}/status, POST /api/v1/events/{eventId}/reprocess, GET /api/v1/events/stats/latest
Metrics (CRFD-103)	GET /api/v1/metrics/summary, GET /api/v1/metrics/timeseries
Alerts (CRFD-105)	GET /api/v1/alerts, POST /api/v1/alerts/{alertRef}/acknowledge, POST /api/v1/alerts/{alertRef}/resolve
Feedback (CRFD-106)	POST /api/v1/feedback, GET /api/v1/feedback
Health and operations	GET /api/v1/health (readiness), GET /api/v1/health/slo (the business gate), GET /actuator/health
Development support	POST /api/v1/dev/simulate (fails closed: 403 in production)

One error contract for everything, produced by a single exception handler: 400 for a bad request, 404 for an unknown path or resource, 405 for the wrong verb on a real path, 409 for a duplicate key or an illegal transition, 503 for readiness or a breached SLO, and 500 only for a genuinely unexpected failure (logged with a correlation id). Turning client errors into 500 was a real defect in this project, fixed as CRFD-121.

2.7 The health model - and why the badge says "DEGRADED - UP"

This is the single most important design decision in the project, and it is visible on the dashboard, so it is worth explaining before the lab:

Endpoint	Answers	200 when	503 when	Who consumes it
GET /api/v1/health	Can this node serve traffic?	UP or DEGRADED	a hard dependency (the database) is down	Container HEALTHCHECK, Nginx, Jenkins deploy gate, Ansible health check
GET /api/v1/health/slo	Are we meeting the quality objective?	within the objective	the success-rate objective is breached	Monitoring, dashboards, humans
GET /actuator/health	Framework-level liveness and detail	the application context is up	context failure	Diagnostics

The rule the project states as a design principle: never let a business metric decide whether a node is able to serve traffic. We got this wrong twice - first in the application, where a seeded database produced a 0.90 success rate against a 0.95 objective and took a perfectly good container out of rotation and failed a good Ansible deployment, and then again in a health-check playbook that asserted UP only and failed on DEGRADED.

2.8 What was built, in numbers

Measure	Value
Automated tests	32 (6 EventService + 6 AlertService unit tests, 20 API integration tests over a real Spring context and a real database)
Selenium end-to-end journeys	5, each mapped to exactly one frozen capability
Deployment smoke checks	11/11, including a write-then-read round trip
Docker image	crfd:1.0.0 / crfd:latest, 401 MB, multi-stage, JRE-only Alpine, non-root, with a HEALTHCHECK
Provisioning	5 playbook runs, idempotent on the second (changed=0), with a working rollback
Git	22 branches, 19 reviewed pull requests, 1 merge conflict created and resolved, 2 annotated tags
Jenkins pipeline	10 stages, the Selenium gate at stage 5, publish at 7, deploy at 8, provisioning at 10
Documentation	19 numbered documents plus this guide, an evidence pack, and a per-PR review record

3. How we planned and ran it

3.1 The planning approach

Cadence	Two-week sprints, with refinement every week
Inception	Week 1 - one week, dedicated to freezing scope and de-risking decisions
Total	15 weeks
Estimation	Story points on a modified Fibonacci scale (1, 2, 3, 5, 8, 13)
Backlog	One ordered backlog in which the product stories AND the toolchain stories are both visible
Board	One issue per user story, labelled user-story, referenced from every commit
Plan of record	docs/02-agile-plan.md; the change log is the Git history

Keeping the toolchain in the same backlog as the product matters. If the Selenium story is in the sprint, the sprint is not done until the journeys run in the pipeline. Infrastructure work is not allowed to be invisible, because invisible infrastructure work is exactly the work that slips.

3.2 Definition of Ready - what may enter a sprint

- It maps to exactly one frozen MVP capability (CRFD-101 to CRFD-106) or to one named toolchain deliverable.
- Its acceptance criteria are written as observable statements, not as intent.
- It fits inside one sprint, or it has been split.
- Its dependencies are already merged, or the dependency is inside the same sprint.
- For a toolchain story, the evidence it will produce is named up front.

3.3 Definition of Done - every line must be true

- Code: implemented on a type/CRFD-<issue>-<description> branch and merged to develop through a pull request that used the PR template.
- Review: the diff has been read against the pull-request reviewer checklist; no unresolved comments.
- Tests: acceptance criteria covered by an automated test, and the test build is green.
- End to end: if the change touches a critical journey, the relevant Selenium journey is updated and green.
- Contract: any API change is reflected in the OpenAPI description and in docs/05-api-catalogue.md.
- Container: the image still builds and the container still reports healthy.
- Docs: CHANGELOG.md updated; affected documents updated.

- Evidence: the artefact that proves it (test summary, screenshot, log, pipeline output) committed under docs/evidence/ or linked from the pull request.
- No secrets and no absolute local paths committed.

3.4 The sprint plan, with the review outcome of each sprint

Sprint	Weeks	Goal	Stories	Review outcome
S0 Inception	1	Agree what is built and how it will be judged	CRFD-100 scope freeze and backlog; CRFD-110 repository skeleton, branch/commit rules, issue and PR templates; two spikes (JAR vs WAR, H2 vs MySQL)	Scope frozen at six capabilities. Two expensive-to-reverse decisions de-risked. The repository, not a document, became the deliverable.
S1 Ingestion	2-3	Events can be published, are idempotent, and are classified	CRFD-101, 101b, 101c, 101d, CRFD-111	POST returns 201 + Location; a repeated eventId returns 409 AND raises a DUPLICATE_EVENT alert - the rejection is itself evidence. The first transaction-boundary bug surfaced here (writes dispatched from an afterCommit hook were silently discarded) and was fixed with REQUIRES_NEW.
S2 Search	4-5	An engineer can find one event among many	CRFD-102, 102b; CRFD-107 part 1	The 5000-row export cap was agreed and enforced, because an unbounded CSV is a denial-of-service vector disguised as a feature.
S3 Indicators	6-7	The dashboard answers "is the pipeline healthy?" before any digging	CRFD-103, 103b; CRFD-107 part 2; CRFD-112	The summary is computed with aggregate queries, not by loading events into memory - invisible at 180 rows, fatal at 180 million.

S4 Drill-down	8-9	An engineer can trace one recommendation	CRFD-104, 104b; CRFD-113; CRFD-114	Every status transition appends an audit entry; the timeline is ordered by (createdAt, id) because two transitions can share a timestamp and the display order must still be stable.
S5 Exceptions	10-11	Nothing fails silently	CRFD-105, 105b; CRFD-107 part 3; CRFD-115	A repeat breach of the same code on the same event does not create a second open alert - because a queue that fills with duplicates gets ignored, and an ignored queue is worse than no queue.
S6 Feedback and E2E	12-13	Quality is measured explicitly and the UI is proven by a real browser	CRFD-106; CRFD-116; CRFD-117; CRFD-130	J2 and J4 failed on the first full run, and they were right to. See Section 5.14: the write path minted an opaque UUID business key while the rest of the system used EVT- prefixed keys.
S7 Release	14-15	Ship it, prove it, and write down what is wrong with it	CRFD-118, 119, 120, 121, 122, 123; CRFD-131, 132, 133	The whole chain runs end to end, and the second run of the provisioning playbook reports changed=0.

3.5 Velocity and the shape of the burndown

Observed velocity settled at roughly 15 points per sprint after sprint 2, at which point the later sprints stopped being re-planned. The burndown is deliberately not smooth: a sprint made mostly of API work completes in one visible step, while a sprint containing a UI story looks flat for several days and then drops.

The useful observation is that no sprint was carried over - which is a consequence of the Definition of Ready, not of heroics: stories whose acceptance criteria were not observable were not allowed into a sprint in the first

place. The per-story point values are in the backlog table in docs/02-agile-plan.md, section 4 of this document uses the same values.

3.6 The DevOps lifecycle, made concrete

Stage	What it is in this project	The artefact that proves it
Plan	Frozen scope, ordered backlog, definition of done, sprint plan	docs/01, docs/02, the issue templates
Code	Feature branches, Conventional Commits, pull requests with review	The Git history, .github/, docs/06
Build	Maven wrapper pinned at 3.9.16, deterministic resolution, profiles for selenium/war/mysql	pom.xml, mvnw, .mvn/
Test	12 unit + 20 API tests + 5 Selenium journeys, with failure screenshots	target/surefire-reports, target/failsafe-reports, docs/evidence/
Release	Two annotated tags, the JAR archived and checksummed by the pipeline	v0.1.0-mvp, v1.0.0, artifact-checksums.txt
Package	Multi-stage Docker image, JRE-only runtime, non-root user	Dockerfile, crfd:1.0.0
Deploy	Jenkins deploys a fresh container; Ansible does the same thing over SSH	Jenkinsfile stages 4/7/8, playbooks/deploy.yml
Operate	Container lifecycle, Nginx in front, comprehensive rollback	scripts/docker-lifecycle.*, playbooks/rollback.yml
Monitor	Readiness and SLO endpoints, the exception queue, scheduled health checks	HealthService, playbooks/healthcheck.yml

The loop closes back to Plan: the defects the test stage found (CRFD-117, CRFD-121, CRFD-117b) became backlog items with their own regression tests. That is the only version of "monitor" that changes anything.

4. Week-wise task plan (weeks 1 to 15)

Two-week sprints after a one-week inception. The "Task" column lists the work items delivered in that week or weeks, using the same identifiers as the backlog and as the Git commit trailers. The "Pts" column is the story-point value of each story from the backlog table.

Week(s)	Sprint	Focus	Tasks delivered	Points
1	S0 Inception	Agree what is built and how it will be judged	CRFD-100: freeze the scope at six capabilities, write the ordered backlog, agree the Definition of Done, choose the stack. CRFD-110: repository skeleton, .gitignore, README, branch naming and commit rules, issue and pull-request templates. Spike 1: Maven profile layout for JAR versus WAR. Spike 2: H2 file mode versus MySQL.	5 + 3
2-3	S1 Ingestion	Events can be published, are idempotent, and are classified	CRFD-101 event ingestion API (POST returns 201 + Location, server mints the EVT- business key). CRFD-101b idempotency: a retried eventId is rejected with 409 and raises a DUPLICATE_EVENT alert. CRFD-101c bulk ingestion with per-item outcomes. CRFD-101d classification into PROCESSED / FAILED / QUARANTINED with an audit row per transition. CRFD-111 Maven build with the wrapper pinned and the selenium / war / mysql profiles.	8 + 3 + 3 + 5 + 3

4-5	S2 Search	An engineer can find one event among many	CRFD-102 searchable dashboard: free text, facets (status, source, surface, type, item, user), paging, sorting. CRFD-102b CSV export of the current filter, capped at 5000 rows. CRFD-107 part 1: the event table, the filter bar and the pager in the UI.	8 + 2
6-7	S3 Indicators	The dashboard answers "is the pipeline healthy?" first	CRFD-103 summary indicators: totals, processed count, success rate, p95 latency, click-through and dismiss rates, breakdowns by status, source and surface. CRFD-103b hourly and daily throughput time series. CRFD-107 part 2: KPI strip, donut and breakdown charts. CRFD-112 multi-stage Dockerfile and the Compose stack with Nginx in front.	8 + 3 + 5
8-9	S4 Drill-down	An engineer can trace one recommendation	CRFD-104 event detail with the immutable audit timeline and related exceptions. CRFD-104b audited manual status transition and reprocess. CRFD-113 Docker lifecycle script with captured evidence (build, tag, run, logs, exec, stop, start, restart, remove). CRFD-114 Jenkins controller as code: Compose +	5 + 3 + 3 + 5

			JCasC + pinned plugins + a seed job.	
10-11	S5 Exceptions	Nothing fails silently	CRFD-105 automatic exception raising from threshold breaches (low confidence, high latency, duplicate, schema mismatch, stale event, rating drop), with per-(code,event) suppression. CRFD-105b the exception queue: filter, acknowledge, resolve, each audited. CRFD-107 part 3: the exception view in the UI. CRFD-115 Jenkinsfile stages through packaging and report publishing.	5 + 5 + 5
12-13	S6 Feedback and E2E	Quality is measured explicitly and the UI is proven by a browser	CRFD-106 explicit feedback capture (rating 1-5 plus reason) and listing by item. CRFD-116 the five Selenium journeys J1-J5, runnable through Maven, with screenshot + page source + JUnit text on failure. CRFD-117 the deliberate defect found by Selenium, fixed, and the suite re-run green. CRFD-130 the unit and API test suite completed (32 tests).	5 + 8 + 3 + 5
14-15	S7 Release	Ship it, prove it, and write down what is wrong with it	CRFD-118 the Selenium quality gate wired into the pipeline, blocking deployment. CRFD-119 Docker build,	3 + 5 + 8 + 2 + 5 + 2 + 3 + 3 + 5

			tag, publish and deploy stages. CRFD-120 Ansible roles and playbooks: provision, deploy, healthcheck, rollback. CRFD-121 the HTTP 500 defect fixed with regression tests. CRFD-122 provisioning evidence: idempotency (changed=0) and rollback. CRFD-123 annotated tags, release notes and the changelog. CRFD-131 end-to-end acceptance walkthrough of all six capabilities. CRFD-132 troubleshooting guide, limitations and future enhancements. CRFD-133 the report pack and viva preparation.	

Two stories deliberately span sprints because they are continuous work rather than a slice: CRFD-107 (the dashboard UI, 8 points, sprints 2-6) and CRFD-130 (the test suite, 5 points, sprints 1-6). Both are reported in the sprint where they finished, which is why the sprint totals in docs/02 are not simply "the points of the stories listed".

4.1 The same plan, as a one-page summary

```
  WEEK  1       freeze the scope, set up the repository and the rules       -> the
repository is the deliverable
  WEEK  2-3     ingest an event, reject a duplicate, classify the outcome   -> 201, 409 +
alert, QUARANTINED
  WEEK  4-5     find one event among many                                   -> q + facets
+ paging + CSV export
  WEEK  6-7     is the pipeline healthy?                                    -> KPIs,
rates, p95, time series
  WEEK  8-9     what happened to THIS recommendation?                       -> detail +
audit timeline
  WEEK 10-11    is anything failing silently?                               -> alert
queue with ack/resolve
  WEEK 12-13    is quality measured, and does a browser agree?              -> ratings +
5 journeys green
```

```
  WEEK 14-15     ship it and write down what is wrong                          -> tags,
gate, Ansible, limitations
```

5. Task-by-task record: what was asked, what was done, what it proves

Each task below follows the same five questions: what the brief asked for, how it was planned, how it was carried out, how it was verified, and where the evidence is. The final row, "Show it in the lab", points at the numbered step in Section 6 that demonstrates it.

5.1 Inception: freeze the scope and set up the repository (CRFD-100, CRFD-110)

What was asked	A frozen MVP scope expressed as an Agile backlog, sprint plan, Definition of Done and a DevOps lifecycle diagram; plus a repository with README, .gitignore, a sensible folder structure, issue templates and branch naming rules.
How it was planned	A one-week inception sprint (Sprint 0). Two spikes were run in parallel with the paperwork because both decisions would be expensive to reverse: the Maven profile layout for JAR versus WAR, and H2 file mode versus MySQL.
How it was carried out	Six capabilities were frozen and written down with observable acceptance criteria; the exclusions were named with their reasons (docs/01). The backlog was written as three epics - Epic A (the product), Epic B (the platform and delivery toolchain) and Epic C (assurance) - all in one ordered list (docs/02). The repository was created with .gitignore, .gitattributes (line-ending policy), README, BRANCHING.md, CONTRIBUTING.md, CODEOWNERS, a user-story issue template, a bug template and a pull-request template.
How it was verified	Every plan document was cross-checked against the delivered tree: each deliverable named in docs/00 maps to a path that exists. The DoD is the same checklist that appears in the issue template, so the definition and the tracking cannot drift apart.
Evidence	docs/01-problem-definition-and-scope.md, docs/02-agile-plan.md, .github/, BRANCHING.md, CONTRIBUTING.md, README.md
Show it in the lab	Step 0 (before you start) and Step 13 (the repository layout and the history).

The decision that mattered: the brief lists two of its deliverables twice. We merged each duplicate into one deliverable and recorded the mapping, rather than delivering the same work twice and calling it coverage.

5.2 Requirements, architecture, data model and API contract (docs/03, docs/04, docs/05)

What was asked	A chosen stack, build tool, database and deployment target with a working local setup; and evidence that the design was thought through rather than improvised.
How it was planned	The architecture was written before the code: functional requirements FR-1 to FR-7 and non-functional requirements NFR-1 to NFR-12 in docs/03, four tables in docs/04, and every endpoint with its request, response and error shape in docs/05. The traceability table at the end of docs/03 maps each capability to its requirement, its API, its automated tests and its Selenium journey.
How it was carried out	A strictly layered monolith: web (controllers, one uniform error contract) to service (ingestion, processing, alerts, metrics, feedback, health) to persistence (Spring Data JPA) to one database. Ten design decisions were recorded with the alternatives that were rejected and why (docs/03 section 4.4) - for example rejecting a bundler to keep the artefact a single JAR, and rejecting one health endpoint in favour of readiness plus a business SLO gate.
How it was verified	The traceability table is checkable: every capability in docs/03 section 5 names real test methods that exist in the suite, and the API catalogue matches the running Swagger UI.
Evidence	docs/03-srs-and-architecture.md, docs/04-data-model.md, docs/05-api-catalogue.md, and http://localhost:8080/swagger-ui/index.html in the running application
Show it in the lab	Step 3 (open Swagger UI next to docs/05 and compare one endpoint) and Step 4 (drive those endpoints).

5.3 Event ingestion: the pipeline and its contracts (CRFD-101, 101b, 101c, 101d, 111)

What was asked	Data / event entry: the ability to publish recommendation feedback events so the rest of the system has something to reason about, built with a chosen build tool.
How it was planned	Split into four slices in Sprint 1 so each one could be finished and tested independently: the single-event API, idempotency, bulk ingestion, and classification with an audit trail. The Maven build (wrapper pinned at 3.9.16, with the selenium, war and mysql profiles) was the platform

	story in the same sprint.
How it was carried out	POST /api/v1/events validates the payload, mints an EVT- business key when the caller does not supply one, persists the event as RECEIVED with an audit row, and returns 201 with a Location header. Processing is dispatched from a commit hook into a REQUIRES_NEW transaction that simulates serving latency, decides PROCESSED / FAILED / QUARANTINED, appends another audit row and evaluates the alert thresholds. A payload missing itemId is quarantined and raises SCHEMA_MISMATCH - losing data silently is worse than losing it loudly, because a silent loss is a bug report you never receive.
How it was verified	Six unit tests on EventService and six API tests covering ingest, bulk, duplicate rejection, invalid payload, quarantine and the key shape; plus the live smoke test, which asserts the 201 plus Location round trip against the container.
Evidence	src/main/java/com/crfd/dashboard/service/EventProcessingService.java, docs/05 section 1, src/test/java/.../EventApiTest.java, docs/evidence/smoke-test.txt
Show it in the lab	Step 4 (ingest the sample event and read the id back) and Step 5 (the duplicate and the quarantine).

```
curl -s -X POST $BASE/api/v1/events -H 'Content-Type: application/json' \
  -d '{"userId":"U-1001","itemId":"ITM-4471","itemTitle":"Noise cancelling headphones",
       "source":"COLLABORATIVE_FILTER","surface":"HOME_FEED",
       "eventType":"CLICK","score":0.87,"latencyMs":210}'
# -> 201, Location: /api/v1/events/EVT-7F3A91C4
```

A defect found in this sprint and fixed here: processing was dispatched from an afterCommit hook, which runs after the original transaction has committed and the persistence context is closed - so without its own transaction the pipeline writes were silently discarded. The fix was REQUIRES_NEW, and the lesson is recorded in docs/11.

5.4 Searchable dashboard and CSV export (CRFD-102, CRFD-102b)

What was asked	A searchable dashboard: find and filter events so an engineer can get to the one they care about.
How it was planned	One sprint slice for search (free text plus facets plus paging plus sorting) and a small second slice for export, because an unbounded export is a different risk from a query.
How it was carried out	GET /api/v1/events takes a free-text q over indexed columns, facet filters for status, source, surface, event type, item and user, a whitelist of sort fields, and paging. Page size is clamped to 1-200 so a client cannot ask for an unbounded page. GET /api/v1/events/export streams the current filter as CSV, capped at 5000 rows and disclosed as capped in the response headers.
How it was verified	API tests for search, facets, paging, sorting and export; the smoke test searches for the event it just wrote,

	which is the only property that makes a search trustworthy - writing and reading are the same rows.
Evidence	docs/05 sections 1.3 and 1.4, src/test/java/.../EventApiTest.java, docs/evidence/smoke-test.txt (the write-then-read check)
Show it in the lab	Step 4 (search by item id, then by status and source) and the dashboard search box in Step 3.

5.5 Summary indicators and time series (CRFD-103, CRFD-103b)

What was asked	Summary indicators so an analyst can see the pipeline vital signs and know whether to dig.
How it was planned	One slice for the indicators and one for the throughput series, both in Sprint 3, with the UI part of the dashboard story in the same sprint so the numbers were visible as soon as they existed.
How it was carried out	GET /api/v1/metrics/summary computes totals, processed count, success rate, p95 latency, click-through and dismiss rates, and breakdowns by status, source and surface - all as aggregate queries, not by loading events into memory. GET /api/v1/metrics/timeseries returns hourly or daily points over a configurable window.
How it was verified	API tests assert the indicators against a known fixture; the p95 is computed in SQL and its portability is listed honestly as a limitation (docs/16, B4).
Evidence	docs/05 section 2, src/main/java/com/crfd/dashboard/service/MetricsService.java, the KPI strip on the running dashboard
Show it in the lab	Step 4 (summary and time series by curl) and Step 3 (the same numbers rendered as the KPI strip and charts).

The decision worth explaining out loud: aggregate queries versus iterating rows in Java is invisible at 180 rows and fatal at 180 million - which is the kind of choice an examiner is listening for.

5.6 Status drill-down and the audited transition (CRFD-104, CRFD-104b)

What was asked	Status drill-down: an engineer must be able to see what happened to one recommendation.
How it was planned	One slice for detail plus timeline, and one for the audited manual transition and reprocess, both in Sprint 4.
How it was carried out	GET /api/v1/events/{eventId} returns the event, its

	immutable audit timeline and the exceptions it raised. Every status change - automatic or manual - appends an audit entry carrying who, why and when. The timeline is ordered by (createdAt, id) because two transitions can share a timestamp and the displayed order must still be stable. PATCH .../status performs an audited manual transition and POST .../reprocess re-runs the pipeline for a failed or quarantined event.
How it was verified	API tests assert the drill-down, the manual transition, that the transition is audited, that an illegal transition is rejected, and that an unknown id returns 404 rather than 500.
Evidence	docs/04 section 3 (the status life cycle), docs/05 sections 1.5 to 1.7, docs/evidence/smoke-test.txt (the drill-down check)
Show it in the lab	Step 4 (the audit trail by curl) and Step 3 (click a row, open the drawer, read the timeline).

5.7 The alert and exception queue (CRFD-105, CRFD-105b)

What was asked	An alert/exception view: a work queue of exceptions so that nothing is silently ignored.
How it was planned	One slice for automatic raising from threshold breaches, one for the queue itself (filter, acknowledge, resolve), both in Sprint 5, with the UI part in the same sprint.
How it was carried out	Threshold evaluation runs at the end of processing and raises alerts for low confidence, high latency, high dismiss rate, duplicate events, schema mismatch, stale events and rating drops. A repeat breach of the same code on the same event does NOT open a second alert while one is already open - suppression is keyed on (code, eventRef) for OPEN alerts only, so a re-breach after a resolve does raise. Acknowledge and resolve each append to the audit trail, and acknowledging an already-resolved alert is rejected.
How it was verified	Unit tests on AlertService cover raising, suppression, the stale flag and the full lifecycle including the illegal transitions; the smoke test exercises the queue against the running container.
Evidence	src/test/java/.../AlertServiceTest.java, docs/05 section 3, docs/evidence/smoke-test.txt
Show it in the lab	Step 4 (list the open alerts, acknowledge one) and Step 5 (the duplicate that raises a DUPLICATE_EVENT alert).

The reasoning to have ready: an alert queue that fills with duplicates gets ignored, and an ignored queue is worse than no queue. Suppression is not laziness - it is the difference between a queue people work and a queue people mute.

5.8 Explicit feedback (CRFD-106, added by change request)

What was asked	The brief lists five capabilities. Rating capture is arguably implied by "feedback dashboard"; we added it as a sixth capability with a change request rather than quietly expanding the scope.
How it was planned	One slice in Sprint 6: capture a rating with a reason, list feedback by item, and raise a RATING_DROP alert on a low rating.
How it was carried out	POST /api/v1/feedback stores a 1-5 rating with an optional reason and the submitter; a low rating raises a RATING_DROP alert against the item. Feedback is deliberately NOT idempotent: the newest rating wins, because a user changing their mind is not a duplicate - recording the first rating and refusing the second would store an opinion the user has already abandoned.
How it was verified	API tests for capture, listing and the low-rating path; the distinction between event idempotency and feedback non-idempotency was argued and settled in pull request #6 and is recorded in the review file.
Evidence	docs/05 section 4, docs/pull-requests/README.md (PR #6), the feedback form on the dashboard
Show it in the lab	Step 4 (submit a rating by curl) and Step 3 (the feedback panel on the dashboard).

5.9 The dashboard user interface (CRFD-107)

What was asked	The MVP has to be a working web application - not just an API. The brief's capabilities are all described from a user's point of view.
How it was planned	Deliberately spread across sprints 2 to 6, one view per sprint, so each capability became visible in the UI as soon as its API existed. Estimated at 8 points and reported in the sprint in which it finished.
How it was carried out	A single page in vanilla ES2020 with no bundler and no framework: index.html, styles.css and app.js served from src/main/resources/static. It renders the KPI strip, the throughput chart, the source breakdown, the searchable event table with facets and paging, the drill-down drawer with the audit timeline, the exception

	queue with acknowledge and resolve, the feedback panel, an event-submission modal, and a health pill. Identity attributes (data-event-id, data-alert-ref, data-value, data-action) were added deliberately so that the Selenium suite can address the thing it means rather than the fourth cell in a row.
How it was verified	All five Selenium journeys drive this UI in a real browser - the dashboard is not taken on trust. Step 3 of Section 6 shows the same page open.
Evidence	src/main/resources/static/, docs/10-selenium-test-plan.md (the locator strategy), docs/pull-requests/README.md (PR #7)
Show it in the lab	Step 3 (the whole page) and Step 8 (the five journeys driving it).

5.10 Git and GitHub collaboration (branching, commits, reviews, conflict, tags)

What was asked	README, .gitignore, folder structure, issue templates, branch naming rules, meaningful commits, feature branches, pull requests with review, a merge conflict created and resolved, and tagging with a release baseline.
How it was planned	A two-branch model: main holds released code only, develop integrates reviewed work. Every story is a feature/CRFD-<n>-<description> branch, merged with --no-ff so the reviewed unit survives in the graph. Branch naming, the commit convention and the review checklist were written down before the first feature branch existed (BRANCHING.md, CONTRIBUTING.md).
How it was carried out	Work was taken from the issue, done on a feature branch, and integrated through a pull request whose description carries a verification table (what was changed, how it was tested, what evidence was produced). Commits use Conventional Commits with the story id in the scope, for example "fix(CRFD-117): mint EVT- prefixed business keys on every write path", and every commit references the issue. Review comments are real: at least three review questions changed the code - the feedback idempotency question (PR #6), the rollback direction question (PR #17), and the alert suppression refinement (PR #4).
How it was verified	The graph is checkable from the command line: git log --first-parent develop lists the reviewed units, git log --graph shows that each branch was merged with two parents, and git for-each-ref lists the annotated tags

	with their tagger dates.
Evidence	docs/06-git-and-github-workflow.md, docs/pull-requests/README.md (19 pull requests with their merge revisions), BRANCHING.md
Show it in the lab	Step 13 (the history, the merge commit with two parents, and the tags).

Stated honestly in docs/06 section 9 and in the limitations: the history is a documented reconstruction. The application and the toolchain were built and verified first, and the history was then laid down against the finished tree, branch by branch, with the defects reintroduced and fixed on their own branches. What is genuine and checkable: every diff, every branch, every merge commit and both of its parents, the conflict, the tags, and the regression tests failing against the pre-fix tree.

5.11 The merge conflict, created and resolved (CRFD-103 versus CRFD-105 documentation)

What was asked	A merge conflict created and resolved, with the resolution explained.
How it was planned	Staged deliberately on comment-only lines in src/main/resources/application.yml, in the same thresholds block: one branch documents why thresholds are overridable, the other documents what a breach does. Both branches were cut from the same tip so the conflict was guaranteed, and it is a genuine content conflict, not an artificial file clash.
How it was carried out	The two branches were both committed, then merged. Git reported "CONFLICT (content): Merge conflict in src/main/resources/application.yml". The two sides were read for intent before any text was touched and were found to be complementary rather than contradictory, so both intentions were kept and rewritten into one statement instead of being concatenated. No threshold value was changed by the resolution.
How it was verified	The suite was re-run after the resolution because a resolution is code nobody wrote: green. The conflict markers were counted and are zero in the committed file, and the merge commit records both parents, so the history shows a real merge rather than a rewrite.
Evidence	docs/evidence/merge-conflict/merge-conflict-output.txt, docs/07-mvp-and-merge-conflict.md section 6, the merge commit e054ea2 with its two parents
Show it in the lab	Step 13 (show the conflict transcript and the merge commit, then show the resolved file).

The point to make in the viva: a conflict resolution is code that nobody wrote, so it has to be re-tested. That is why the suite was re-run before the merge was pushed.

5.12 The release baseline: tags, changelog and release notes (CRFD-123)

What was asked	Tagging and a release baseline.
How it was planned	Two annotated tags rather than lightweight ones, because a tag should carry authorship and intent: v0.1.0-mvp at the point where all six capabilities were demonstrable, and v1.0.0 at the release merge. The release was done on a release/v1.0.0 branch so the changelog could be brought up to date and the state frozen long enough to verify, then merged into main with --no-ff and back into develop so develop is never behind main.
How it was carried out	The CHANGELOG records both baselines with real dates and the defects fixed (CRFD-117, CRFD-117b, CRFD-121), the alert-suppression change (CRFD-105) and a security note about the simulator failing closed. Before the tag was cut, a release verification pass was run against the frozen tree - build, tests, smoke test and Selenium - and two documentation defects found during that pass were fixed on the release branch.
How it was verified	The tag points at a state that was individually verified; git show v1.0.0 prints the release message, and git tag -l lists both tags. Every revision cited in the report was checked to resolve to a real commit.
Evidence	CHANGELOG.md, docs/07 section 5, docs/evidence/release-verification.txt
Show it in the lab	Step 13 (git tag -l, git show v1.0.0) and the changelog file itself.

Release verification is the step that found CRFD-117b - business keys being minted in the shape of a truncated UUID rather than the documented eight-hex-character form. No test had caught it, because no test had asserted the documented shape; one does now.

5.13 Jenkins: installation and configuration (CRFD-114)

What was asked	Jenkins installed and configured, pointed at the repository, building when a commit appears, with the build artefact archived.
How it was planned	As code, in Sprint 4. A controller a human clicked together cannot be reviewed, reproduced or rebuilt, so the controller is a Compose stack plus Configuration as

	Code (JCasC), a pinned plugin list and a seed job: infra/jenkins/docker-compose.jenkins.yml, jenkins.yaml, plugins.txt and the seed job definition.
How it was carried out	The stack starts the controller on port 8085 together with a local Docker registry on 5001, joins the pinned network crfd-ci-net, and mounts the host Docker socket so the controller can start sibling containers. JCasC applies the configuration on boot, and the Job DSL seed job creates the pipeline job that reads the Jenkinsfile from the repository, so the pipeline definition lives in source control rather than in the controller.
How it was verified	The controller starts, applies JCasC and creates the job; this is recorded as a configuration demonstration. What was NOT done is stated plainly in the limitations: the full pipeline was never orchestrated end to end by Jenkins, because that needs a real GitHub remote to check out from. Every stage command was run and captured by hand instead.
Evidence	infra/jenkins/, docs/08-jenkins-ci-job.md, docs/16 limitation A3
Show it in the lab	Step 11 (start the controller, show the seed job and the job parameters). If the controller is not started, show infra/jenkins/jenkins.yaml and the seed job definition instead.

This is the honest weak point of the deliverable and it is stated in the first paragraph of the limitations rather than buried: Jenkins orchestrating the whole chain in one click is not something this project can claim.

5.14 The Jenkinsfile: ten stages, and the parameterised environment (CRFD-115, 118, 119)

What was asked	A Jenkinsfile with checkout / build / package / deploy stages and at least one parameterised environment setting; and a failing test that stops the deployment.
How it was planned	Ten stages, with the ordering decided by one rule: everything to the right of the quality gate is irreversible, so the gate must be to its left. That rule was learned from a real bug (see the note below).
How it was carried out	The stages are: 1 checkout and metadata; 2 build and unit test; 3 package and checksum; 4 Docker build and tag; 5 Selenium end-to-end quality gate; 6 publish test reports; 7 publish the image; 8 deploy a fresh container; 9 health check and smoke test; 10 provision the node with Ansible. The image is built once (stage 4) and the same tag is gated (5), pushed (7) and deployed (8), so the gate can never validate one artefact while

	another ships. Parameters include ENVIRONMENT, APP_PORT, IMAGE_TAG, REGISTRY, ANSIBLE_INVENTORY, RUN_SELENIUM, PUBLISH_IMAGE, DEPLOY, RUN_ANSIBLE and the two environment settings CRFD_LOW_CONFIDENCE and CRFD_CRITICAL_LATENCY_MS - which is the brief's "parameterised environment setting", demonstrated by the same image alerting on different thresholds per environment. In production the pipeline also forces simulation and seeding off, because synthetic traffic and a seeded database are what make a demo pleasant and a production deployment wrong. A commit triggers the build through pollSCM, polling every five minutes with H/ spreading the load; the job is also parameterised for on-demand builds and rehearsals.
How it was verified	Stage ordering was verified by inspection after fixing the ordering bug, and the networking between job containers was verified independently - the containerised Selenium suite reached the application at http://crfd-app:8080 by network alias, never localhost.
Evidence	Jenkinsfile, docs/09-pipeline-and-deployment.md, docs/13 sections 2 and 3, docs/08 sections 6 to 8
Show it in the lab	Step 9 (the gate and what it blocks) and Step 11 (the pipeline definition and its parameters).

```
pipeline {
  stages {
    stage('1 Checkout & metadata')           { ... }
    stage('2 Build & unit test')             { ... }   // 32 tests
    stage('3 Package')                       { ... }   // jar + sha256
    stage('4 Docker build & tag')            { ... }   // crfd:v<version>-<build>,
crfd:<sha>, crfd:latest
    stage('5 Selenium E2E quality gate')     { ... }   // <- the gate
    stage('6 Publish test reports')          { ... }
    stage('7 Publish image')                 { ... }   // <- irreversible from here
    stage('8 Deploy fresh container')        { ... }
    stage('9 Health check & smoke test')     { ... }
    stage('10 Provision node (Ansible)')     { ... }
  }
}
```

The bug worth telling: the first draft built the image AFTER the gate, so the gate tried to run an image that did not exist. The fix was not just reordering - it was understanding why the order was wrong. A gate that validates one artefact while a different artefact ships is theatre: it consumes the time of running the tests and returns none of the confidence.

5.15 The deployment target: Tomcat and Nginx (CRFD-112, CRFD-119)

What was asked	A Tomcat/Nginx deployment target with a working local setup.
How it was planned	Both halves of the requirement were kept buildable

	rather than choosing one and arguing: embedded Tomcat as the default (because a container wants one file and one process), an external Tomcat 10.1 path through the war profile, and Nginx in front in the Compose stack.
How it was carried out	The default build produces an executable fat JAR with embedded Tomcat. The war profile produces target/crfd.war for an external Tomcat 10.1. In the Compose stack Nginx terminates on port 80, proxies / to the application, exposes /healthz, applies rate limiting to the ingest path, and is held behind depends_on: service_healthy so it cannot come up in front of an application that is not ready.
How it was verified	The Compose stack was started and /healthz answered through Nginx; the WAR was built and deployed to a Tomcat container and the application was reachable under /crfd/.
Evidence	Dockerfile, docker-compose.yml, docs/09 sections 2 and 7, docs/12 section 5
Show it in the lab	Step 14 (optional: the WAR on an external Tomcat and the Nginx edge).

If Nginx never starts, read the troubleshooting guide (docs/17 section 3.1) before anything else: it is the project signature failure, and the cause is the readiness-versus-SLO distinction explained in Section 2.7.

5.16 Selenium: the five critical journeys (CRFD-116)

What was asked	3-5 critical user journeys run locally through Maven, with screenshots on failure, integrated into Jenkins, and a failure that stops the deployment.
How it was planned	Five journeys, each mapped to exactly one frozen capability, so a red journey immediately names the capability at risk. Journeys were chosen to cover capabilities rather than code paths. They run through the Maven selenium profile using the failsafe plugin, in two interchangeable modes: a containerised Chrome (the way the pipeline runs them) or a local browser.
How it was carried out	J1 loads the dashboard and asserts the KPI tiles, the health pill, the environment badge, both charts, the source breakdown and the event table. J2 submits a feedback event and asserts the confirmation contains an EVT- business key, that the modal closes, that the row appears, and that the search box is re-populated with that key - proving the UI wired the new event into the filter. J3 combines free text with the status facet and asserts the rows and the pager agree. J4 drills into

	an event, asserts the header id starts with EVT-, that the timeline has at least three entries, that the related exception is present, and that closing the drawer hides it. J5 lists open alerts, acknowledges the newest, and re-filters by ACKNOWLEDGED to find the same alert by reference. Failures write three things: a screenshot, the page source, and the JUnit failure text.
How it was verified	5/5 green, twice, against the containerised application with a containerised browser. More importantly, the suite is known to be able to fail: it was run against the pre-fix tree and four regression tests failed - the only property that makes a test worth keeping.
Evidence	src/test/java/.../selenium/, docs/10-selenium-test-plan.md, docs/evidence/selenium-remote-run.log, docs/evidence/regression-tests-fail-before-fix.txt
Show it in the lab	Step 8 (run the five journeys live) and Step 9 (the failure evidence and the gate).

```
docker run -d --name crfd-selenium -p 4444:4444 -p 7900:7900 \
    --network crfd-net selenium/standalone-chrome:4.25.0

$env:SELENIUM_REMOTE_URL = "http://localhost:4444/wd/hub"
.\mvnw.cmd -B -P selenium failsafe:integration-test failsafe:verify \
    "-Dselenium.baseUrl=http://crfd-app:8080" "-Dselenium.headless=true" "-
Dselenium.browser=chrome"
# -> Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Two networking facts that caused real confusion before they were understood: SELENIUM_REMOTE_URL is reached from the host, so localhost is correct there; selenium.baseUrl is reached from inside the browser container, so it must be a Docker network alias - localhost inside that container is the container itself.

5.17 Continuous testing, the defects it caught, and the fixes (CRFD-117, 117b, 121, 130)

What was asked	A deliberately introduced defect that a test catches, fixed, with the suite re-run green - plus the test suite that makes that possible.
How it was planned	Defects were planned as first-class backlog items with the same definition of done as features: defect, root cause, fix, regression test, evidence. The regression tests live on their own branch so every intermediate commit still builds.
How it was carried out	CRFD-117: the event write paths minted opaque UUIDs while the rest of the system used EVT- prefixed business keys. J2 and J4 failed, and the screenshot plus page source made the cause obvious in minutes. Fixed on a bugfix branch; the suite went green. CRFD-121: client errors (an unknown path, the wrong verb) were

	returned as HTTP 500 instead of 404 and 405; fixed with regression tests for both. CRFD-117b: found by release verification rather than by a test - keys were being minted in the shape of a truncated UUID (for example EVT-B1CB5C50-F67) instead of the documented eight-hex-character form; a contract test now asserts the shape, and the fix landed on the release branch before the tag was cut. Two further corrections came out of this work: the readiness versus SLO split (Section 2.7) and the anonymous-volume trap in the deploy (Section 5.18).
How it was verified	Each defect has a failing-before and passing-after pair: the regression tests fail against the pre-fix tree (docs/evidence/regression-tests-fail-before-fix.txt) and pass afterwards; the release verification file records the smoke test printing a malformed key before the repair and a well-formed one after it.
Evidence	docs/11-continuous-testing-and-defect-fix.md, docs/evidence/regression-tests-fail-before-fix.txt, docs/evidence/selenium-defect-crfd-117/, docs/evidence/release-verification.txt
Show it in the lab	Step 9 (the failing-before evidence, then the green run) - this is the strongest five minutes of the demonstration.

The distinction to make clearly: a defect caught by a test proves the test works. A defect caught by verification rather than by a test (CRFD-117b) is a gap in the tests, and the honest response is to add the assertion that was missing - which is what the contract test does.

5.18 Docker: the image and the container lifecycle (CRFD-112, CRFD-113)

What was asked	A Dockerfile, build/tag/run, port mapping, logs, and the stop/restart/remove lifecycle.
How it was planned	A multi-stage build so the build toolchain does not ship: a build stage with the JDK and the Maven cache, and a runtime stage that is JRE-only Alpine running as a non-root user. The container exposes a named data volume and a HEALTHCHECK, and the image carries its own identity as build arguments.
How it was carried out	The image builds to 401 MB with three tags: crfd:1.0.0 (the release, immutable by convention, the one that deploys), crfd:<git-sha> (the only tag that unambiguously answers "which commit is this?"), and crfd:latest (a convenience pointer that is never deployed). The container is run with the port mapped 18080 to 8080, the named volume crfd-data mounted at /app/crfd-data, and the environment set through

	variables. The lifecycle - run, logs, health, exec, stop, start, restart, remove - is exercised deliberately by scripts/docker-lifecycle.* and captured, because the point of running it once on purpose is to find out now that restart preserves the volume mount, rather than during an incident.
How it was verified	11/11 smoke checks against the running container, an inspect showing the health status and the volume mount, and a deliberate demonstration that data survives a container removal because the volume is named.
Evidence	Dockerfile, scripts/docker-lifecycle.*, docs/12-docker-lifecycle.md, docs/evidence/docker-images.txt, docs/evidence/docker-inspect.txt, docs/evidence/smoke-test.txt
Show it in the lab	Step 6 (build, run, smoke test) and Step 7 (the lifecycle), then Step 10 (image identity).

The trap worth telling: the image declares VOLUME /app/crfd-data, so running it without -v creates an ANONYMOUS volume, which docker rm then destroys. Replace-and-run would have wiped the event history on every deployment. Both the pipeline and the Ansible role mount the named volume explicitly.

5.19 Jenkins to Docker continuous deployment (CRFD-119)

What was asked	Jenkins must build a versioned image, publish it to Docker Hub or a local registry, and deploy a fresh container.
How it was planned	Stages 4, 7 and 8 of the Jenkinsfile, with the gate in between (stage 5), plus a local registry on port 5001 in the Jenkins Compose stack so the publish can be demonstrated without credentials.
How it was carried out	Stage 4 builds and tags the image with the resolved version tag, the git sha, and latest. Stage 7 pushes the version tag and the sha tag through docker.withRegistry. Stage 8 removes the previous container and starts a fresh one from the gated tag with the named data volume mounted, then stage 9 polls the health endpoint and runs the write-then-read smoke test against the deployed container. Everything the pipeline archives is chosen to answer a question later: the fingerprinted JAR answers "which bytes were built", artifact-checksums.txt answers "is this the same artefact", the image inspect answers "what is inside the image", the Selenium reports and screenshots answer "did the gate really run", and the deployed container log answers "what did the application say".

How it was verified	The individual stage commands were executed and captured; the pipeline definition and its ordering were verified by inspection. A full Jenkins-orchestrated run is not claimed - see the limitations.
Evidence	Jenkinsfile, docs/13-jenkins-docker-cd.md, docs/08 section 8 (the archived-artefact table)
Show it in the lab	Step 10 (image identity and the three tags) and Step 11 (the pipeline stages as defined in the repository).

Networking rule to state out loud when asked how Jenkins reaches what it deploys: Jenkins runs as a container using the host Docker socket, so the containers it starts are siblings. localhost inside a build step is Jenkins itself. Everything is addressed by network alias on the pinned network: crfd-app, crfd-selenium, crfd-live.

5.20 Ansible: provisioning a clean node, idempotency and rollback (CRFD-120, CRFD-122)

What was asked	Puppet or Ansible: provision a clean node, prove idempotency, health-check it, and demonstrate rollback/recovery.
How it was planned	Ansible was chosen over Puppet for one honest reason: the project central claim is "here is a clean node, now watch it be provisioned", and Ansible is agentless, so the node genuinely starts with SSH and nothing else. Puppet is better at fleet scale and would need an agent installed first, which weakens the demonstration. The roles are common, docker, crfd_app, and the playbooks are site.yml, deploy.yml, healthcheck.yml and rollback.yml.
How it was carried out	A disposable clean node (SSH plus a Docker engine, nothing else) and a pinned Ansible controller are started by one command. site.yml provisions the node, deploy.yml moves the release and records the rollback point, healthcheck.yml verifies the node is serving, and rollback.yml restores the recorded release. The rollback record is written only when the version changes - a task that rewrites a file on every run can never be idempotent. The record is written from the release being left, so after a rollback the node can roll forward again: recovery is symmetric, because a rollback that leaves a node unable to move forward is a trap dressed as a fix.
How it was verified	Five captured runs: step 1 ok=30 changed=9; step 2, the identical run, ok=30 changed=0 - idempotency, as Ansible own count of the tasks that did anything; step 3 deploy ok=20 changed=4; step 4 healthcheck ok=7 changed=0 reporting UP with the SLO within objective;

	step 5 rollback ok=23 changed=3 ending "RECOVERED crfd-node -> crfd:0.1.0, health=UP". The release history log shows both transitions with timestamps.
Evidence	infra/ansible/, scripts/ansible-demo.*, docs/14-ansible-provisioning.md, docs/evidence/ansible-01..05-*.log and ansible-summary.txt
Show it in the lab	Step 12 (run the whole demonstration with -KeepRunning, then read previous-release.txt and release-history.log on the node).

```
powershell -ExecutionPolicy Bypass -File scripts\ansible-demo.ps1 -KeepRunning

  step 1  site.yml -e app_version=0.1.0      ok=30 changed=9  failed=0
  step 2  site.yml -e app_version=0.1.0      ok=30 changed=0  failed=0   <- idempotent
  step 3  deploy.yml -e app_version=1.0.0    ok=20 changed=4  failed=0
  step 4  healthcheck.yml                    ok=7  changed=0  status=UP
slo=WITHIN_OBJECTIVE
  step 5  rollback.yml                       ok=23 changed=3  RECOVERED crfd-node ->
crfd:0.1.0
```

changed=0 is the deliverable. It is not a claim in a document; it is Ansible own count of the tasks that did anything, and the answer is none. Have this sentence ready before you are asked.

5.21 Assurance: walkthrough, troubleshooting, limitations, report and viva (CRFD-131, 132, 133)

What was asked	An end-to-end workflow, a complete report, a troubleshooting guide, limitations, future enhancements, and a presentation/viva.
How it was planned	Four documents as four backlog items in the release sprint, written against the delivered system rather than in advance of it.
How it was carried out	docs/15 is the end-to-end walkthrough with the observed results and a viva question bank. docs/16 is the limitations list, split into limitations of the exercise (A1-A9) and limitations of the product (B1-B9), with a prioritised future-enhancements list. docs/17 is the troubleshooting guide - 30 real failures, each with the message it produced, the cause and the fix, including the Windows/PowerShell quirks that cost the most time. docs/18 is the local runbook that reproduces every figure, and this document is the explanation and demonstration guide built on top of it.
How it was verified	Every cross-reference between documents was checked to resolve, every revision cited was checked to exist, and every figure quoted was re-measured against the repository rather than copied forward.
Evidence	docs/15, docs/16, docs/17, docs/18, this document,

	docs/evidence/release-verification.txt
Show it in the lab	Step 0 (the document pack as the first thing on the desk) and the closing two minutes: limitations stated before they are asked for.

Saying the weakest part first is a deliberate strategy, not modesty: a reviewer who finds an unstated weakness stops believing the rest of the report.

6. What to show in the lab: the step-by-step demonstration

This is the run sheet. It assumes a Windows machine with PowerShell, JDK 21, Docker Desktop in Linux-container mode, and the project folder. Each step states the goal, the exact command, the result to expect, the sentence to say while it runs, and what to fall back on if it does not run. The whole sequence takes about 20 minutes; the six-minute version is Steps 2, 3, 4, 6 and 9.

Golden rule for the lab: never let the demonstration depend on something you did not run five minutes ago. Every step below has a committed evidence file behind it, so if a live command misbehaves you show the captured output and say plainly that you are showing the captured run instead. That is stronger than a pause.

The 60-second version (if you only get one shot)

```
cd C:\Users\mrudu\Desktop\content-recommendation-feedback-dashboard

.\mvnw.cmd -B clean package            # 32 tests
docker build -t crfd:1.0.0 .
docker run -d --name crfd-app -p 18080:8080 -v crfd-data:/app/crfd-data crfd:1.0.0
powershell -ExecutionPolicy Bypass -File scripts\smoke-test.ps1 -BaseUrl
http://localhost:18080   # 11/11
start http://localhost:18080/          # the dashboard
```

Step 0 - the five minutes before you start

Have open	This document (Section 6), a terminal in the project folder, a browser at http://localhost:18080/, and a file explorer on docs\evidence\
Have ready	docs/evidence/release-verification.txt (the whole verification in one file) and docs/evidence/selenium-defect-crfd-117/ (the failing screenshots)
Check nothing collides	Port 8080 is frequently taken by something unrelated. The container is published on 18080 for exactly that reason. The project uses 8080 (app), 18080 (app container), 80 (Nginx), 4444/7900 (Selenium), 8085 (Jenkins), 5001 (registry), 3306 (MySQL).
Know the first sentence	"This is a working dashboard, and everything around it - the tests, the image, the pipeline, the provisioning - is automated and has evidence committed."

Step 1 - prove the toolchain in four commands (30 seconds)

Goal: show that the environment the project claims is the environment you have, before anything else can be blamed on it.

```
java -version                                    # must say 21
.\mvnw.cmd -v                                    # Maven 3.9.16 via the wrapper
docker version --format "server={{.Server.Version}} os={{.Server.Os}}"   # os must be
linux
git --version
```

Expected: Java 21, Maven 3.9.16, Docker server in Linux mode, Git present.

Say	"JDK 21, and Maven is pinned by the wrapper, so my laptop, the container and CI all use the same build tool - that is the point of committing mvnw."
If it fails	Java 17 or 8 will give UnsupportedClassVersionError later; fix JAVA_HOME first (docs/17 section 1.1). Windows containers cannot run the Alpine image - switch Docker Desktop to Linux containers.

Step 2 - build and run the 32 automated tests (2 minutes)

Goal: the product compiles and its unit and API contract tests pass, on your machine, from a clean build.

```
.\mvnw.cmd -B clean package
[INFO] Tests run: 6,  Failures: 0, Errors: 0, Skipped: 0  -- AlertServiceTest
[INFO] Tests run: 6,  Failures: 0, Errors: 0, Skipped: 0  -- EventServiceTest
[INFO] Tests run: 20, Failures: 0, Errors: 0, Skipped: 0  -- EventApiTest
[INFO] Tests run: 32, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Say	"12 unit tests on the domain services and 20 API tests over a real Spring context and a real database. The five browser journeys are deliberately not in this run - they need a browser, and they are Step 8."
Show the artefact	target\content-recommendation-feedback-dashboard.jar (that is the runnable fat JAR; the name comes from the Maven finalName)
If it fails	Run offline once dependencies are cached: .\mvnw.cmd -B -o clean package. If the failure is a version error, it is Step 1 again.

Optional, if asked about coverage: `.\mvnw.cmd -B test jacoco:report` then open target\site\jacoco\index.html.

Step 3 - run the application and open the dashboard (2 minutes)

Goal: show the MVP as a user sees it. This is the step that answers "is it a real working application?"

Either run it from source, or use the container that is already running (Step 6 explains the container):

```
# from source
java -jar target\content-recommendation-feedback-dashboard.jar

# and then open
start http://localhost:8080/          # or http://localhost:18080/ for the container
```

Expected: Tomcat starts on 8080 within about ten seconds, and the dashboard renders the header, the health pill, the KPI strip, the throughput chart, the source breakdown, the searchable event table and the exception list.

Walk through it	1) The KPI strip: events, processed, success rate, failures, open exceptions, latency, click-through. 2) Search for an item id. 3) Click a row to open the drawer and read the audit timeline. 4) Open the exception queue and acknowledge one. 5) Submit a new event from the modal and watch it appear.
Say about the badge	"The badge says DEGRADED and UP at the same time, and that is deliberate: this node can serve traffic, but the pipeline success rate is below its objective." Then hit /api/v1/health and /api/v1/health/slo to show 200 and 503 from the same healthy application.
If it fails	Port 8080 in use: use an already-published container on 18080, or set SERVER_PORT. Nginx never starting is covered in Step 14 and docs/17 section 3.1.

Useful URLs to leave open: dashboard /, readiness /api/v1/health, the business gate /api/v1/health/slo, the API catalogue /swagger-ui/index.html, and the database console /h2-console.

Step 4 - drive the API end to end (4 minutes)

Goal: prove the six capabilities are real contracts, not screenshots. Five short curl calls, in the order a user would work.

```
BASE=http://localhost:18080

# 1. ingest an event - the server mints an EVT- business key
curl -s -X POST $BASE/api/v1/events -H 'Content-Type: application/json' \
  -d '{"userId":"U-1001","itemId":"ITM-4471","itemTitle":"Noise cancelling headphones",
       "source":"COLLABORATIVE_FILTER","surface":"HOME_FEED",
       "eventType":"CLICK","score":0.87,"latencyMs":210}'
# -> 201 + Location: /api/v1/events/EVT-XXXXXXXX     (that key shape is a contract)

# 2. find it again
curl -s "$BASE/api/v1/events?q=ITM-4471"
curl -s "$BASE/api/v1/events?status=PROCESSED&source=COLLABORATIVE_FILTER&size=5"

# 3. drill in - the status history is the point
curl -s $BASE/api/v1/events/EVT-XXXXXXXX          # .status and .auditTrail

# 4. the vital signs
curl -s "$BASE/api/v1/metrics/summary?windowHours=24"
curl -s "$BASE/api/v1/metrics/timeseries?granularity=hour&windowHours=6"
```

```
# 5. the exception queue and explicit feedback
curl -s "$BASE/api/v1/alerts?status=OPEN"
curl -s -X POST $BASE/api/v1/feedback -H 'Content-Type: application/json' \
  -d '{"itemId":"ITM-4471","rating":2,"reason":"too expensive","submittedBy":"U-1001"}'
```

Say	"Six capabilities: entry, search, indicators, drill-down, exceptions, explicit feedback. Each one has an endpoint, a test and, for five of them, a browser journey."
If it fails	A 400 means the payload used the wrong field names (the contract is userId, itemId, itemTitle, eventType, source, surface, score). Show docs/05 instead and move on.

Step 5 - the two behaviours that look like bugs and are contracts (2 minutes)

Goal: show that the system makes deliberate choices under failure. This is the step that separates a project from a tutorial.

```
# 1. idempotency: the second post is rejected AND recorded
curl -s -X POST $BASE/api/v1/events -H 'Content-Type: application/json' \
  -d '{"eventId":"EVT-FIXED0001","userId":"U-1","itemId":"ITM-1","source":"TRENDING",
       "surface":"HOME_FEED","eventType":"CLICK","score":0.5}'
curl -s -o /dev/null -w '%{http_code}\n' -X POST $BASE/api/v1/events -H 'Content-Type:
application/json' \
  -d '{"eventId":"EVT-FIXED0001","userId":"U-1","itemId":"ITM-1","source":"TRENDING",
       "surface":"HOME_FEED","eventType":"CLICK","score":0.5}'
# -> 409   and a DUPLICATE_EVENT alert is now in the queue

# 2. quarantine, not rejection: a payload with no itemId
curl -s -X POST $BASE/api/v1/events -H 'Content-Type: application/json' \
  -d '{"userId":"U-
1","source":"TRENDING","surface":"HOME_FEED","eventType":"CLICK","score":0.5}'
# -> accepted, stored as QUARANTINED, raises SCHEMA_MISMATCH - and NOT a 500
```

Say	"Events are idempotent because an event is a fact and a fact repeated is a replay; feedback is not idempotent because a user changing their mind is not a duplicate. And a payload missing its item is quarantined rather than rejected, because losing data silently is worse than losing it loudly."
If it fails	Nothing here is timing-dependent; if the second POST returns 201 you are talking to a different instance - check the port.

Step 6 - build the image, run the container, run the 11 smoke checks (3 minutes)

Goal: the same code runs as a container from a single image, with a named volume for its data, and it serves the API correctly.

```
docker build -t crfd:1.0.0 .
docker images crfd                                  # expect ~401 MB

docker run -d --name crfd-app --restart unless-stopped `
    -p 18080:8080 `
    -v crfd-data:/app/crfd-data `
    -e CRFD_ENVIRONMENT=demo `
    -e CRFD_CRITICAL_LATENCY_MS=900 `
    crfd:1.0.0

docker inspect --format "{{.State.Health.Status}}" crfd-app        # -> healthy
docker logs --tail 20 crfd-app                                     # the app's own log

powershell -ExecutionPolicy Bypass -File scripts\smoke-test.ps1 -BaseUrl
http://localhost:18080
# -> 11/11 PASS
```

Say	"The final image is a JRE-only Alpine image running as a non-root user, so nothing at runtime can compile code, and the port is mapped 18080 to 8080. The smoke test is a write-then-read round trip plus the health, metrics and alert endpoints - reachability is not correctness."
Point at the volume	"-v crfd-data:/app/crfd-data is not decoration: the image declares a VOLUME, so without it Docker creates an anonymous volume that docker rm destroys. That would wipe event history on every deploy."
If it fails	If the build needs a rebuild from scratch it takes a few minutes; the image may already exist locally, in which case skip the build. If the smoke test fails on health, read docs/17 section 3.

Step 7 - exercise the container lifecycle deliberately (2 minutes)

Goal: the brief asks specifically for build, tag, run, port mapping, logs, stop, restart, remove. Run it as a reviewed sequence rather than ad hoc.

```
powershell -ExecutionPolicy Bypass -File scripts\docker-lifecycle.ps1
```

Say	"The point of running the lifecycle on purpose is to learn once, calmly, that restart preserves the volume mount and that stop takes a few seconds because shutdown is graceful - rather than discovering either during an incident."
Show	docs/evidence/docker-inspect.txt (the volume mount

	and health) and docs/evidence/docker-images.txt (the tags and size)
If it fails	Run the four commands by hand instead: docker stop / docker start / docker restart / docker rm, then docker run again with the volume.

Step 8 - run the five Selenium journeys (4 minutes)

Goal: a real browser drives the real UI, and the suite is wired into Maven, which is what the brief asks for.

```
docker run -d --name crfd-selenium -p 4444:4444 -p 7900:7900 `
    --network crfd-net selenium/standalone-chrome:4.25.0

$env:SELENIUM_REMOTE_URL = "http://localhost:4444/wd/hub"
.\mvnw.cmd -B -P selenium failsafe:integration-test failsafe:verify `
    "-Dselenium.baseUrl=http://crfd-app:8080" `
    "-Dselenium.headless=true" `
    "-Dselenium.browser=chrome"
# -> Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
```

Name the journeys	J1 the dashboard renders its indicators; J2 a submitted event becomes searchable by its EVT- key; J3 free text plus the status facet agrees with the pager; J4 the drill-down shows the audit timeline; J5 an open alert is acknowledged and found again by reference.
Show it live	Open http://localhost:7900 (VNC, password secret) while the suite runs - the examiner can watch the browser click through the application.
Say	"Each journey maps to exactly one capability, so a red journey names the capability that broke. The locators use data-* identity attributes rather than CSS position, so styling changes do not break the suite."
If it fails	A version mismatch between Chrome and its driver is docs/17 section 5.3. If a journey is flaky, that was a real bug in this project (a stale element during auto-refresh) and it is fixed - docs/17 section 5.1 explains why the fix was a re-locate rather than a longer sleep.

Step 9 - show that the gate can fail, and what a failure blocks (3 minutes)

Goal: this is the strongest evidence in the project. A test suite that has never failed is a script, not a gate.

```
# the failing run, preserved
docs\evidence\selenium-defect-crfd-117\
    J2_-_...-20261018-174501.png     what the user saw
    J2_-_...-20261018-174501.html    what the DOM actually contained
    J2_-_...-20261018-174501.txt     what the assertion wanted
```

```
# and the proof that the tests can fail at all
docs\evidence\regression-tests-fail-before-fix.txt
```

Tell the story	The write path minted an opaque UUID while everything else used EVT- prefixed keys. J2 and J4 failed. The screenshot, the page source and the JUnit text made the cause obvious in minutes, and the fix is one commit: fix(CRFD-117): mint EVT- prefixed business keys on every write path.
Say what it blocks	"In the Jenkinsfile the gate is stage 5 of 10, and publish and deploy are stages 7 and 8. A red gate means the image is never published and the running container is never replaced - the ordering is the whole argument of the pipeline."
Offer the follow-up	"Release verification then found a second, smaller version of the same defect - keys shaped like a truncated UUID. No test had asserted the documented shape; now one does. A defect found by verification rather than by a test is a gap in the tests, and the honest response is to add the missing assertion."
If it fails	Nothing to fail: this step shows committed evidence files. If asked to reproduce the failure, check out the parent of the fix commit and run the regression tests - docs/07 section 6.5 gives the procedure.

Step 10 - image identity and versioning (2 minutes)

Goal: the brief asks for a versioned image; show what versioning has to mean and that the image carries its own identity.

```
docker inspect crfd:1.0.0 --format '{{json .Config.Env}}'
# ["APP_VERSION=1.0.0-SNAPSHOT","BUILD_REVISION=local", ...]

docker image ls --digests crfd
# crfd  1.0.0  sha256:ce0912682d08 ...      the release tag (deploys)
# crfd  <sha>                                 the only tag that says which commit this is
# crfd  latest                                a convenience pointer, never deployed
```

Say	"latest is a name, not a version - it points at different bytes on different days, which makes rollback meaningless. The pipeline builds three tags for one image and deploys the version tag, and it bakes APP_VERSION and BUILD_REVISION in as build arguments, so docker inspect answers the question docker ps cannot: which source revision produced this container."
If it fails	Show the same facts from docs/evidence/docker-images.txt and docs/evidence/docker-inspect.txt.

Step 11 - Jenkins as code (3 minutes)

Goal: Jenkins is installed and configured, the job exists, the build triggers on a commit, and the artefact is archived - all reproducible from the repository.

```
docker compose -f infra/jenkins/docker-compose.jenkins.yml up -d
docker logs -f crfd-jenkins          # watch JCasC apply; about 60 s on first run
# then open http://localhost:8085

docker network inspect crfd-ci-net --format '{{range .Containers}}{{.Name}} {{end}}'
docker images | Select-String registry        # the local registry on 5001
```

Show in the repository	infra/jenkins/jenkins.yaml (Configuration as Code), plugins.txt (pinned plugin versions), the seed job that creates the pipeline job, and the Jenkinsfile itself - the pipeline definition lives in source control, not in the controller.
Say	"The controller is disposable: delete the container, run compose up, and the same controller with the same job comes back. That is the difference between installed and configured."
Be honest here	"The pipeline needs a real GitHub remote to check out from, which this environment does not have, so I ran every stage command individually and verified the ordering by inspection. Jenkins orchestrating all ten stages in one click is not something I can claim - it is the first limitation in my report."
If it fails	If the controller is not running, show jenkins.yaml, plugins.txt and the seed job definition in the editor, plus the Jenkinsfile stage list. Same evidence, no container.

Step 12 - provisioning: clean node, idempotency, health, rollback (5 minutes)

Goal: the brief asks for a clean node brought to a working state, with idempotency evidence, a health check and rollback. One command runs all five steps and writes the evidence.

```
powershell -ExecutionPolicy Bypass -File scripts\ansible-demo.ps1 -KeepRunning
step 1  site.yml -e app_version=0.1.0      ok=30 changed=9  failed=0
step 2  site.yml -e app_version=0.1.0      ok=30 changed=0  failed=0     <- idempotency
step 3  deploy.yml -e app_version=1.0.0    ok=20 changed=4  failed=0
step 4  healthcheck.yml                    ok=7  changed=0  status=UP slo=WITHIN_OBJECTIVE
step 5  rollback.yml                       ok=23 changed=3  RECOVERED crfd-node ->
crfd:0.1.0, health=UP
```

Then, with -KeepRunning, inspect the node itself. This is the part examiners remember:

```
docker exec crfd-node cat /etc/crfd/previous-release.txt       # 1.0.0
docker exec crfd-node cat /etc/crfd/release-history.log        # both transitions,
timestamped
docker exec crfd-node wget -qO- http://localhost:8080/api/v1/health
```


Say slowly	"changed=0 on the second identical run is not a claim in a document - it is Ansible own count of the tasks that did anything, and the answer is none."
Say about the rollback	"The rollback record is written from the release being left, so after rolling back the node can roll forward again. Recovery is symmetric - a rollback that leaves a node unable to move forward is a trap dressed as a fix."
Say about Ansible vs Puppet	"Ansible, because it is agentless: the node genuinely starts with SSH and nothing else, which is what makes the demonstration a demonstration. Puppet is better at fleet scale and would need an agent first - the trade-off is in docs/14."
If it fails	The five captured logs are in docs/evidence/ansible-01..05-*.log with ansible-summary.txt. Show the summary and the healthcheck log.

Step 13 - the Git history, the conflict and the tags (3 minutes)

Goal: the collaboration requirements - branches, reviewed pull requests, a merge conflict, tags - shown from the repository, not from a slide.

```
git log --oneline --graph --decorate -25          # the branch and merge shape

git log --first-parent --oneline develop | Select-Object -First 12
# -> the list of reviewed units, one per pull request

git log --format="%h %p %s" -1 e054ea2            # the conflict merge: two parents
git tag -l                                        # v0.1.0-mvp, v1.0.0
git show v1.0.0 | Select-Object -First 12         # the annotated tag message

git log --oneline -1 c97ae88                      # fix(CRFD-117): mint EVT- keys on every
write path
```

Show the conflict	docs/evidence/merge-conflict/merge-conflict-output.txt - CONFLICT (content) in src/main/resources/application.yml, both sides kept because the two comments were complementary, and the suite re-run green afterwards because a resolution is code nobody wrote.
Say about the model	"main holds released code only and develop integrates reviewed work. Every story is a feature/CRFD-<n>-<description> branch merged with --no-ff, so git log --first-parent reads as the list of reviewed units - 19 of them - and git bisect still lands on a single commit."
Volunteer the caveat	"The commit dates are all from one reconstruction session rather than spread across fifteen weeks, and the reviewer role was taken by the same person. What is genuine and checkable is every diff - including the defects, which really were reintroduced and really did

	fail the tests - every branch, both parents of every merge commit, the conflict, and the tags."
If it fails	docs/06 sections 8 to 10 and docs/07 section 6 contain the same facts as transcripts.

Step 14 - optional extras, if there is time (3 minutes)

Use these only if the main run finished early. They answer the "deployment target" and "database" parts of the brief most directly.

```
# a) the WAR on an external Tomcat 10.1 (the brief names Tomcat)
.\mvnw.cmd -B -P war clean package
docker run -d --name crfd-tomcat -p 18090:8080 `
    -v "$PWD\target\crfd.war:/usr/local/tomcat/webapps/crfd.war" `
    tomcat:10.1-jdk21
start http://localhost:18090/crfd/

# b) MySQL instead of H2, to show the schema and queries are portable
docker compose -f docker-compose.mysql.yml up -d
$env:SPRING_PROFILES_ACTIVE = "mysql"
java -jar target\content-recommendation-feedback-dashboard.jar

# c) Nginx in front, with the health probe
docker compose up -d
docker compose ps
curl -s -o /dev/null -w '%{http_code}\n' http://localhost/healthz     # -> 200, from Nginx
```

Say	"Embedded Tomcat is the default because a container wants one file and one process; the external Tomcat path stays buildable because the brief names it. Nginx is in front so the entry point survives a container replacement, and it holds the app behind depends_on: service_healthy so it cannot come up in front of an application that is not ready."
If it fails	Nginx 502 is docs/17 section 4.5, and it is almost always the readiness-versus-SLO mistake.

Step 15 - teardown (30 seconds, only when finished)

```
docker rm -f crfd-app crfd-selenium crfd-node crfd-live crfd-jenkins 2>$null
docker rm -f crfd-tomcat crfd-mysql registry 2>$null
docker network rm crfd-net crfd-ci-net crfd-demo-net 2>$null
docker volume rm crfd-data 2>$null            # <- this deletes the event history
docker rmi crfd:1.0.0 crfd:latest crfd:0.1.0
docker compose -f docker-compose.yml down
docker compose -f infra/jenkins/docker-compose.jenkins.yml down
```

Do not run teardown before the viva is over, and never run docker compose down -v on the machine you are demonstrating from: it removes the named volume that holds the event history.

Step 16 - if a live step fails: the fallback table

Step	What it demonstrates	Committed evidence to show instead
2 build and tests	the suite is green	docs/evidence/smoke-test.txt, docs/evidence/release-verification.txt (build + 32 tests)
4 API tour	the six capabilities	docs/05-api-catalogue.md plus the captured smoke test
5 contracts	409 and quarantine	the ingestion tests named in docs/03 section 5
6 container	image, port mapping, volume, health	docker-images.txt, docker-inspect.txt, smoke-test.txt
7 lifecycle	stop/start/restart/remove	the docker-lifecycle transcript
8 Selenium	five journeys green	docs/evidence/selenium-remote-run.log
9 gate	the gate can fail and blocks deploy	selenium-defect-crfd-117/ and regression-tests-fail-before-fix.txt
11 Jenkins	the controller is configured as code	infra/jenkins/jenkins.yaml, plugins.txt, the seed job, Jenkinsfile
12 Ansible	idempotency and rollback	ansible-summary.txt and the five step logs
13 Git	branches, conflict, tags	docs/06, docs/07, docs/pull-requests/README.md, merge-conflict-output.txt

7. Evidence index

Everything in docs/evidence/ was produced by a command in the runbook (docs/18 section 12). Nothing in this table is a claim; each row is a captured output or a captured artefact.

File or folder	Produced by	What it proves
smoke-test.txt	scripts/smoke-test.ps1 against the container	11/11 checks, including a write-then-read round trip and the health, metrics and alert endpoints
health-and-slo.txt	curl to /api/v1/health and /api/v1/health/slo	Readiness returns 200 while the business gate can return 503 - the two are separate concerns
docker-images.txt	docker images crfd	The tags and the 401 MB size
docker-inspect.txt	docker inspect crfd-app	The health status, the port mapping and the named-volume mount
docker-runtime-probe.txt	a probe inside the container	The runtime is a JRE only - nothing can compile Java at runtime
selenium-remote-run.log	the Maven selenium profile against the container	5/5 journeys green with a containerised browser
selenium-defect-crfd-117/	the first failing run of J2	A screenshot, the page source and the JUnit text - the model for triaging any journey failure
regression-tests-fail-before-fix.txt	the suite against the pre-fix tree	Four regression tests fail without the fixes - proof the tests can fail
merge-conflict/merge-conflict-output.txt	the staged merge of the two documentation branches	A real content conflict, the markers as git printed them, and the resolved state
ansible-01-provision-first-run.log	scripts/ansible-demo.* step 1	A clean node becomes a serving CRFD node: ok=30 changed=9 failed=0
ansible-02-idempotency-second-run.log	the identical run again	changed=0 - idempotency, as Ansible own count
ansible-03-deploy-release.log	step 3, deploy 0.1.0 to 1.0.0	A release move with the rollback point recorded
ansible-04-healthcheck.log	step 4	status=UP with the SLO within objective, and it changes nothing (changed=0)
ansible-05-rollback.log	step 5	RECOVERED crfd-node -> crfd:0.1.0 with health=UP
ansible-summary.txt	the five runs, condensed	The one-page answer to the provisioning requirement
release-verification.txt	a full release verification pass	The build, the 32 tests, the smoke test and the five journeys against the frozen tree, plus the two report findings

The repository also carries process evidence outside docs/evidence/: docs/pull-requests/README.md records all 19 pull requests with their reviews and merge revisions, and CHANGELOG.md records both release baselines.

8. Troubleshooting quick reference

These are failures that actually happened during this project, with the message they produced. The full guide, with 30 entries and the reasoning behind each fix, is docs/17-troubleshooting-guide.md.

Symptom	Cause	Fix
UnsupportedClassVersionError on startup	The runtime is not Java 21 (the project compiles to class file 65)	Set JAVA_HOME to the Temurin 21 install and reopen the shell
Port 8080 already in use	Something unrelated holds the port	Use the published container on 18080, or set SERVER_PORT
409 on an event POST	The eventId was already used - this is the idempotency contract, not a failure	Expected: the response is recorded and a DUPLICATE_EVENT alert is raised
An event never leaves RECEIVED	Processing is asynchronous and you are reading too early	Set CRFD_ASYNC_ENABLED=false for deterministic, same-request processing
An event is QUARANTINED with SCHEMA_MISMATCH	The payload had no itemId	Expected behaviour, deliberately not a rejection and not a 500
A 404 or 405 used to return 500	Historical defect CRFD-121 - a catch-all handler swallowed client errors	Fixed, with regression tests for both the unknown path and the wrong verb
The container is unhealthy but the application serves everything	A business metric was deciding availability	Readiness is /health (200 for UP or DEGRADED); the objective is /health/slo. The rule: never let a business metric decide whether a node can serve traffic
/api/v1/health/slo returns 503 in a good deployment	The success rate is genuinely below the objective - the endpoint is doing its job	Nothing to fix. Check CRFD_MINIMUM_SUCCESS_RATE before assuming a bug
Deploying wipes all the data	The image declares VOLUME and the run had no -v, so an anonymous volume was destroyed	Always mount the named volume: -v crfd-data:/app/crfd-data
docker: unable to apply cgroup configuration (nested node)	A nested Docker engine cannot apply cgroup controllers	Clear the container resource limits for the demo - this is why they are opt-in
Nginx starts and returns 502	Nginx came up in front of an application that was not ready, or readiness is failing	Check /api/v1/health first, then docs/17 section 3.1 - the project signature failure
StaleElementReferenceException, one run in five	The dashboard auto-refreshes, so a read can lose its element	Re-locate and retry on that exception. A longer sleep masks the problem and slows every run
Connection refused inside the browser container	selenium.baseUrl used localhost, which is the browser container itself	Use the Docker network alias: http://crfd-app:8080
A Jenkins step cannot reach localhost	Jenkins is a sibling container, not a host process	Address the application by network alias on crfd-ci-net: crfd-app, crfd-selenium, crfd-live

The pipeline parameters do not appear in the job	The seed job redeclared the job and overrode the Jenkinsfile	The Jenkinsfile owns the parameters; the seed job deliberately does not redeclare them
Ansible: the role common was not found	The role path was relative to the controller, not to the playbook	Fix the roles_path, then confirm alignment with ansible-lint --syntax-check
Ansible: the second run is not idempotent	A task rewrites a file on every run	Write the rollback record only when the version changes
The demo node cannot enforce CPU and memory limits	Nested Docker cannot apply cgroup controllers	Clear app_memory_limit and app_cpu_limit for the demo run
git push rejected as non-fast-forward	The remote has commits the local branch does not	Fetch and rebase, or merge - the same conflict discipline applies
A .sh file arrives with CRLF after clone	The line-ending policy was not applied on a Windows clone	The .gitattributes policy forces LF for *.sh, mvnw, *.yml, *.java and the Jenkinsfile; re-check out the file
A generated script is blocked as malicious	PowerShell AMSI blocked a long generated script	Generate with a different interpreter or write the file in smaller pieces - this was hit twice during this project

9. Limitations and future enhancements

Stated first rather than last, because a reviewer who finds an unstated weakness stops trusting the rest of the report. The full list with reasoning is docs/16.

9.1 Limitations of the exercise

	Limitation	Why it is stated
A1	The Git history is a reconstruction, not a fifteen-week log	The system was built and verified first and the history laid down against the finished tree; the defects were genuinely reintroduced and genuinely failed the tests
A2	There is no GitHub remote, so the pull requests are recorded rather than opened	The review record, the merge commits and the diffs are real; the web UI is not
A3	The Jenkins pipeline was never run end to end by Jenkins	The controller needs a real GitHub remote to check out from. Every stage command was run and captured individually and the ordering verified by inspection
A4	The reviewer and the author are the same person	Stated plainly rather than presented as peer review
A5	One node, one environment, one host	No fleet, no multi-node rolling deployment
A6	The provisioning target is a nested Docker engine	That is why container resource limits are opt-in for the demo
A7	No load, scale or soak testing	The p95 and throughput figures are therefore indicative, not benchmark results
A8	No security review beyond configuration hygiene	No penetration test, no dependency scanning, no image signing or SBOM
A9	No lint or policy enforcement on the infrastructure code	Ansible and the Dockerfiles are reviewed by reading, not by a linter in the pipeline

9.2 Technical limitations of the product

	Limitation	Consequence
B1	H2 in file mode is single-writer	Fine at MVP volume; MySQL is the multi-writer shape and is what the mysql profile exists for
B2	No schema migration tool	The schema is created by JPA; a real deployment needs Flyway or

		Liquibase before the first breaking change
B3	The % and _ characters act as wildcards in a search term	A search for AB_CD can match more than expected
B4	p95 latency is computed in SQL and is not portable	A different database needs the projection re-checked
B5	Alert suppression can hide a worsening incident	A repeat breach of the same code on the same event does not re-raise while one is open
B6	Feedback is not idempotent	A repeated submission overwrites - deliberate, but it means feedback cannot be replayed
B7	No authentication, authorisation or operator audit	Acceptable only because the MVP is deployed on a trusted internal network
B8	The browser is a third-party image and the journeys are English-only	A locale change would break assertions on text
B9	Accessibility is not verified	Contrast and keyboard navigation follow good practice but are not tested

9.3 Future enhancements, in priority order

11. A real GitHub remote with branch protection and required status checks, so the gate is enforced by the server and not only by the Jenkinsfile.
12. An externalised database with schema migration (Flyway) and a connection-pool story under load, plus multi-node provisioning to exercise serial: 1 and max_fail_percentage for real.
13. Alert suppression refined so that a worsening incident re-raises: suppression on (code, eventRef) becomes suppression on rate or severity escalation.
14. A load test to make the p95 and throughput figures meaningful, and a soak test to find leaks.
15. Delivery integrations for alerts (email, Slack, PagerDuty), which is an integration rather than a capability.
16. Image signing and an SBOM, so the pipeline can answer "what is in this image" as precisely as it answers "which revision is this".

10. Viva: the questions this project invites, and the answers that hold up

Scope and architecture

Question	Answer
Justify the frozen scope. What did you leave out?	Six capabilities frozen in week 1. Left out deliberately: streaming ingestion, multi-tenancy, a real recommendation model, authentication, and alert delivery integrations. The exclusion that needed an argument was explicit feedback - it was added as CRFD-106 with a change request rather than smuggled in, because a frozen scope that quietly grows is not frozen.
Why Spring Boot with an embedded container when the brief says Tomcat?	The deployment target is a container, and a fat JAR with embedded Tomcat is what a container wants: one file, one process, no external container to manage. The Tomcat requirement is met two ways - embedded by default, and a war profile producing crfd.war for an external Tomcat 10.1 - with Nginx in front either way.
What does Nginx actually do here?	Terminates on port 80, proxies to the application, exposes /healthz for tooling, applies rate limiting to the ingest path, and holds the app behind depends_on: service_healthy so it cannot come up in front of something that is not ready.
Why is the front end not React?	A bundler adds a Node toolchain, a second build step, a node_modules directory in CI and a hash manifest to the deploy. The dashboard is one page with five views, so the artefact stays a single JAR.

Data and the API

Question	Answer
An event arrives with no itemId. What happens?	It is quarantined: stored with QUARANTINED status and a SCHEMA_MISMATCH alert. Rejecting it loses data; discarding it loses it silently, which is worse because a silent loss is a bug report you never receive.
Why are events and feedback treated differently on repeat submission?	An event is a fact, and a fact repeated is a replay, so the same eventId twice is a 409 plus a DUPLICATE_EVENT alert. A user changing their mind is not a duplicate, so the newest rating wins - recording the first and refusing the second would store an opinion the user has abandoned.
What does the drill-down show that the KPI strip does not?	The KPI strip answers "is the aggregate healthy"; the drill-down answers "what happened to this one". Both read models are served from the same event rows, which is what makes the drill-down trustworthy: the

	count and the row are literally the same rows.

CI/CD

Question	Answer
A test fails in the pipeline. What happens next?	Nothing is published and nothing is deployed. The gate is stage 5 of 10; publish is 7 and deploy is 8. Demonstrated rather than asserted: the CRFD-117 failure produced no image and no deployment.
Why is the image built before the gate?	It was not, in the first draft, and that was the bug - the gate tried to run an image that did not exist. The fix was not merely reordering: a gate that validates one artefact while a different artefact ships is theatre. The image is now built once and referenced by one tag through build, gate, publish and deploy.
Jenkins is a container. How does it reach the application it deploys?	By network alias on a pinned network, never localhost. Jenkins uses the host Docker socket, so the containers it starts are siblings; everything is on crfd-ci-net as crfd-app, crfd-selenium or crfd-live. Host port publishing exists for humans, not for the pipeline.
What is the parameterised environment setting the brief asks for?	Several, all pipeline parameters - ENVIRONMENT, APP_PORT, IMAGE_TAG, REGISTRY, and the threshold settings CRFD_LOW_CONFIDENCE and CRFD_CRITICAL_LATENCY_MS. The same image alerts on different thresholds per environment, which is the clearest demonstration that configuration is externalised rather than baked in. In production the pipeline also forces simulation and seeding off.
Is the build reproducible?	The wrapper pins Maven 3.9.16, the JAR is built once and checksummed, and the image records APP_VERSION and BUILD_REVISION as build arguments, so docker inspect answers which source revision produced a running container.

Selenium

Question	Answer
How many journeys, and what makes them critical?	Five, chosen to cover capabilities rather than code: the dashboard renders its indicators; a submitted event becomes searchable; search plus a facet agrees with the pager; the drill-down shows the audit timeline; an open alert is acknowledged and found again. Each fails if a whole capability is broken.
Your suite flaked. What did you do?	Found the cause rather than the symptom. The table auto-refreshes, so a read could lose its element between locating it and reading it. The fix re-locates

	and retries on that exception - a longer sleep would have masked it and slowed every run. Making the test reliable is what exposed the real defect.
How do you know your tests can fail?	They were run against the pre-fix tree and four regression tests failed. That is the only property that makes a test worth keeping.

Docker and provisioning

Question	Answer
Why multi-stage, and what does it buy?	The build stage has the JDK and the Maven cache; the runtime stage has a JRE. The image is 401 MB instead of close to a gigabyte, and nothing at runtime can compile Java, so a compromised container cannot build attacker-supplied code with the shipped toolchain.
Your deploy removes the container and starts a new one. Is that not data loss?	Only if the volume is not mounted - and that is a real trap this project hit. The image declares VOLUME /app/crfd-data, so a run without -v creates an anonymous volume that docker rm destroys. Both the pipeline and the Ansible role mount the named volume crfd-data explicitly.
Puppet or Ansible, and why?	Ansible. Both were permitted, but the central claim is "here is a clean node, now watch it be provisioned", and Ansible is agentless, so the node genuinely starts with SSH and nothing else. Puppet is better at fleet scale and would need an agent first, which weakens the demonstration - the trade-off is in docs/14 rather than presented as a clear win.
Prove idempotency.	The second identical run reports changed=0 - Ansible own count of the tasks that did anything, and the answer is none. The mechanism: the rollback record is written only when the version changes; a task that rewrites a file every run can never be idempotent.
What does rollback do, and what if the rollback itself breaks?	rollback.yml restores the release named in /etc/crfd/previous-release.txt, and that file is written from the release being left, so after a rollback the node can roll forward again. Recovery is symmetric; a rollback that leaves a node unable to move forward is a trap dressed as a fix.

The two design corrections (offer these before being asked)

Question	Answer
Which mistake did you make twice?	Letting a business metric control availability. /api/v1/health first returned 503 whenever anything was off, including a success rate below its objective - so

	a seeded database (about 0.90 against 0.95) made a fully functional instance report unhealthy, the container health check failed, Nginx never started, and a good Ansible deployment failed. Split into readiness (/health) and the business gate (/health/slo). Then the health-check playbook made the same mistake, asserting UP only and failing on DEGRADED. The rule now stated as a design principle: never let a business metric decide whether a node can serve traffic.
Why does /health return 200 for DEGRADED?	Because DEGRADED answers a business question and the endpoint answers an availability question. Returning 503 for it is how a reporting number takes a working node out of a load balancer - which is exactly what happened here, twice.

Git, honesty and next steps

Question	Answer
Why --no-ff instead of squash or fast-forward?	Because git log --first-parent develop then reads as the list of reviewed units - 19 merge commits naming the pull request. Squash loses the intermediate commits, so a bisect lands on one enormous commit; fast-forward makes the branch vanish from the graph.
Walk me through the merge conflict.	Two branches cut from the same tip, both editing the same comments above the threshold block. UU on application.yml. The two sides were read for intent and found complementary - one says why thresholds are overridable, the other what a breach does - so both were kept and rewritten into one statement. The suite was re-run because a resolution is code nobody wrote: green. Both parents are in the merge commit, so the history shows a real merge.
What is not fully honest in this repository?	Two things, both stated rather than hidden: the commit dates come from one reconstruction session rather than a fifteen-week calendar, and the reviewer role was taken by the author. Everything checkable is genuine - every diff including the reintroduced defects, every branch, both parents of every merge commit, the conflict, and the tags.
What would you do with another four weeks?	In order: a real GitHub remote with branch protection and required status checks; an externalised database with Flyway migration and a load-tested connection pool; suppression refined so a worsening incident re-raises; multi-node provisioning to exercise serial and max_fail_percentage for real; and a load test to make the latency figures meaningful.
What is the weakest part of this deliverable?	The full Jenkins pipeline was never executed end to end by Jenkins, because the controller needs a GitHub

	remote this environment does not have. Every stage command was run and captured individually and the ordering verified by inspection, but "Jenkins orchestrated it in one click" is not a claim this project can make. It is listed first in the limitations.

11. Appendix

A. Ports and URLs

Port	Used by	URL or note
8080	the application from source (default)	http://localhost:8080/
18080	the application container, published	http://localhost:18080/ - used throughout this document
80	Nginx, in the Compose stack	http://localhost/ and http://localhost/healthz
4444 / 7900	Selenium standalone Chrome	http://localhost:7900 for the live VNC view (password: secret)
8085	the Jenkins controller	http://localhost:8085
5001	the local Docker registry	the pipeline push target
3306	MySQL, when the profile is on	docker-compose.mysql.yml
18090	the external Tomcat demo (war profile)	http://localhost:18090/crfd/

B. Useful application URLs

URL	What it is
/	the dashboard
/api/v1/health	readiness - 200 for UP or DEGRADED, 503 only when a hard dependency is down
/api/v1/health/slo	the business gate - 200 within the objective, 503 when breached
/swagger-ui/index.html	the API catalogue, live
/h2-console	the database console (JDBC URL jdbc:h2:file:./crfd-data/crfd, user sa, empty password)
/actuator/health	framework-level health, for diagnostics

C. Configuration: every operational value is an environment variable

Variable	Default	What it changes
SERVER_PORT	8080	the listening port
CRFD_ENVIRONMENT	local	the environment badge, and what the container believes it is
CRFD_SIMULATION_ENABLED	true	the demo traffic generator; fails closed with 403 when off
CRFD_SEED_ON_STARTUP	true	seeds a realistic dataset at startup

CRFD_SEED_EVENT_COUNT	180	how many events the seeder writes
CRFD_ASYNC_ENABLED	true	false processes on the request thread, which is what makes tests deterministic
CRFD_LOW_CONFIDENCE	0.30	the confidence floor that raises LOW_CONFIDENCE
CRFD_HIGH_LATENCY_MS	500	the high-latency warning threshold
CRFD_CRITICAL_LATENCY_MS	1500	the critical latency threshold
CRFD_MINIMUM_SUCCESS_RATE	0.95	the objective the SLO gate measures against
CRFD_METRICS_WINDOW_HOURS	24	the default window for the indicators

The threshold variables are the brief's "parameterised environment setting": the same image, deployed by the same pipeline, alerts on different thresholds per environment, which is the clearest proof that configuration is externalised rather than compiled in.

D. Command cheat sheet

```
# build, test, package
.\mvnw.cmd -B clean package                                  # 32 tests + the runnable JAR
.\mvnw.cmd -B test jacoco:report                             # coverage report
.\mvnw.cmd -B -P war clean package                           # target\crfd.war for
external Tomcat

# run
java -jar target\content-recommendation-feedback-dashboard.jar

# container
docker build -t crfd:1.0.0 .
docker run -d --name crfd-app -p 18080:8080 -v crfd-data:/app/crfd-data crfd:1.0.0
docker inspect --format "{{.State.Health.Status}}" crfd-app
docker logs -f crfd-app

# verification
powershell -ExecutionPolicy Bypass -File scripts\smoke-test.ps1 -BaseUrl
http://localhost:18080
powershell -ExecutionPolicy Bypass -File scripts\docker-lifecycle.ps1

# the browser suite
docker run -d --name crfd-selenium -p 4444:4444 -p 7900:7900 --network crfd-net
selenium/standalone-chrome:4.25.0
$env:SELENIUM_REMOTE_URL = "http://localhost:4444/wd/hub"
.\mvnw.cmd -B -P selenium failsafe:integration-test failsafe:verify `
    "-Dselenium.baseUrl=http://crfd-app:8080" "-Dselenium.headless=true" "-
Dselenium.browser=chrome"

# pipeline and provisioning
docker compose -f infra/jenkins/docker-compose.jenkins.yml up -d          # Jenkins on
8085
powershell -ExecutionPolicy Bypass -File scripts\ansible-demo.ps1 -KeepRunning

# evidence, whenever a demo step misbehaves
```

```
git log --oneline --graph --decorate -25
git tag -l
```

E. Key revisions, if a reviewer asks to see a specific piece of history

Reference	What it is
v0.1.0-mvp	the annotated MVP tag, cut after pull request #7 - all six capabilities demonstrable
v1.0.0	the annotated release tag - the full lifecycle, commit to provisioned node
the CRFD-117 fix	the one-line-idea fix that minted EVT- business keys on every write path
the CRFD-121 fix	client errors returned as 404 and 405 instead of 500
the conflict merge	the merge commit with two parents produced by the application.yml conflict
the readiness fix	the separation of readiness from the business SLO gate

F. The first sixty seconds, and the last two minutes

First: "This is a working recommendation feedback dashboard - I can ingest an event, find it, drill into its audit timeline and work the exception queue it raises. Around that product there is a full delivery chain: 32 automated tests, five browser journeys, a container image, a Jenkins pipeline whose stage 5 quality gate blocks deployment, and Ansible provisioning that is provably idempotent and can roll back. Everything I show you has a committed artefact behind it, and the things I could not do are written down in the limitations rather than left for you to find."

Last: state the weakest point before it is asked, then the two design corrections - readiness versus the SLO gate, and the anonymous volume - because those are the two places where the project shows judgement rather than effort.